Cloud–Edge–End Collaborative Task Offloading in Vehicular Edge Networks: A Multilayer Deep Reinforcement Learning Approach

Jiaqi Wu, *Graduate Student Member, IEEE*, Ming Tang, *Member, IEEE*, Changkun Jiang, *Member, IEEE*, Lin Gao, *Senior Member, IEEE*, and Bin Cao, *Member, IEEE*

***Abstract*—Mobile-edge computing (MEC) is a promising computing scheme to support computation-intensive AI applications in vehicular networks, by enabling vehicles to offload computation tasks to edge computing servers deployed on road side units (RSUs) that approximate to them. In this work, we consider an MEC-enabled vehicular edge network (VEN), where each vehicle can offload tasks to edge/cloud computing servers via vehicle-to-infrastructure (V2I) links or to other end-vehicles via vehicle-to-vehicle (V2V) links. In such a *cloud–edge–end* collaborative offloading scenario, we focus on the joint task offloading, scheduling, and resource allocation problem for vehicles, which is challenging due to the online and asynchronous decision-making requirement for each task. To solve the problem, we propose a *Multilayer deep reinforcement learning* (DRL)-based approach, where each vehicle constructs and trains three modules to make different layers' decisions: 1) *Offloading Module* (first layer), determining whether to offload each task, by using the dueling and double deep *Q*-network (D3QN) framework; 2) *Scheduling Module* (second layer), determining where and how to offload each task in the offloading queues, together with the transmission power, by using the parameterized deep *Q*-network (PDQN) framework; and 3) *Computing Module* (third layer), determining how much computing resource to be allocated for each task in the computation queues, by using classic optimization techniques. We provide the detailed algorithm design and perform extensive simulations to evaluate its performance. Simulation results show that our proposed algorithm outperforms the existing algorithms in the literature, and can reduce the average cost by 25.86%–72.51% and increase the average satisfaction rate by 3.48%–90.53%.**

***Index Terms*—Deep reinforcement learning (DRL), mobile-edge computing (MEC), task offloading, vehicular edge network (VEN).**

Received 31 May 2024; revised 5 September 2024; accepted 22 September 2024. Date of publication 1 October 2024; date of current version 7 November 2024. This work was supported in part by the National Key Research and Development Program of China under Grant 2021YFB2900300; in part by the National Natural Science Foundation of China under Grant 61972113, Grant 62202214, and Grant 61902255; in part by the Shenzhen Science and Technology Program under Grant KQTD20190929172545139, Grant GXWD20231129103946001, Grant ZDSYS20210623091808025, and Grant JCYJ20190808163417094; in part by the Guangdong Basic and Applied Basic Research Foundation under Grant 2021B1515120008 and Grant 2023A1515012819; and in part by the Natural Science Foundation of Guangdong Province under Grant 2024A1515010178. This article was presented in part at IEEE International Conference on Communications (ICC 2023) [1] [DOI: 10.1109/ICC45041.2023]. *(Corresponding authors: Lin Gao; Bin Cao.)*

Jiaqi Wu, Lin Gao, and Bin Cao are with the School of Electronics and Information Engineering and the Guangdong Provincial Key Laboratory of Aerospace Communication and Networking Technology, Harbin Institute of Technology, Shenzhen 518055, China (e-mail: gaol@hit.edu.cn).

Ming Tang is with the Department of Computer Science and Engineering, Southern University of Science and Technology, Shenzhen 518055, China (e-mail: tangm3@sustech.edu.cn).

Changkun Jiang is with the College of Computer Science and Software Engineering, Shenzhen University, Shenzhen 518060, China (e-mail: jiangchangkun@gmail.com).

Digital Object Identifier 10.1109/JIOT.2024.3472472

I. Introduction

A. Background

With the rapid development of self-driving and automated-driving technologies, a large number of AI-enabled vehicular network applications (such as 3-D visual reconstruction [2] and real-time objective recognition [3]) are emerging. These AI-enabled applications are often resource-hungry, computation-intensive, and latency-sensitive [4]. Traditional cloud computing scheme may *not* be able to support such applications due to large communication overhead and end-to-end latency. *Mobile-edge computing (MEC [5], [6])* is emerging as a promising computing scheme to support AI-enabled applications in future networks, via offloading some computation tasks to edge computing servers deployed on network edge (e.g., base stations) that approximate to end-users. In an MEC-enabled vehicular edge network (VEN), vehicles can offload computation-intensive tasks to edge computing servers deployed on road side units (RSUs), and hence can effectively reduce the task latency and improve the Quality-of-Service (QoS). Therefore, MEC has been widely recognized as one of the key enabling technologies for AI-enabled applications in the future vehicular networks [7], [8], [9], [10], [11].

In an MEC-enabled VEN, one key and important problem is to decide *whether and how to offload each task*, namely, how to choose the appropriate computing entities and wireless transmission links for task offloading. Many existing researches (e.g., [12], [13], [14], [15], [16], [17], and [18]) in this field focused on the *edge/cloud offloading mode*, where tasks from vehicles are offloaded to edge computing serves on RSUs or cloud computing servers through vehicle-to-infrastructure (V2I) transmission links via macro base stations (MBSs). However, these works did not consider the collaboration among end-vehicles, specifically overlooking the under-utilized computing resources of vehicles that could be utilized for other vehicles' tasks. With the fast development

2327-4662 © 2024 IEEE. Personal use is permitted, but republication/redistribution requires IEEE permission.
See https://www.ieee.org/publications/rights/index.html for more information.

Authorized licensed use limited to: Eotvos Lorand University Budapest. Downloaded on October 27,2026 at 13:00:42 UTC from IEEE Xplore. Restrictions apply.

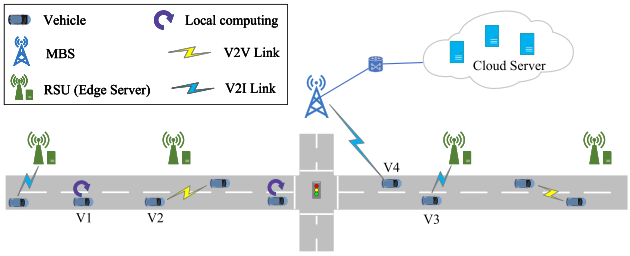


Fig. 1. Illustration of MEC-enabled VEN model.

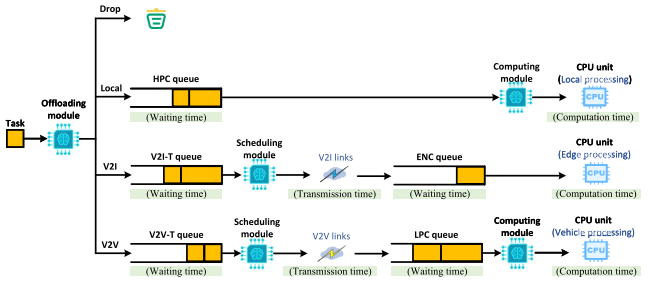


Fig. 2. Illustration of queue-based cloud–edge–end collaborative offloading framework.

of 5G/6G cellular vehicle-to-everything (C-V2X) technology, together with the continuous increase of on-board computing resources on end-vehicles, a new *end-collaborative offloading* mode is emerging in VEN. In this mode, tasks from vehicles can be offloaded to other vehicles with surplus resources through vehicle-to-vehicle (V2V) transmission links [19]. It has been shown that the integration of both offloading modes can significantly enhance system performance, e.g., reducing task latency and alleviating edge network congestion.

Based on the above, it is worth exploring a *cloud–edge–end collaborative offloading* mode that integrates both V2I-enabled edge/cloud offloading and V2V-enabled end collaboration. Some recent works [20], [21], [22], [23], [24] have studied the cloud–edge–end collaborative task offloading problem by using classic *optimization-based* approaches. However, optimization-based approaches often have poor generalization capabilities, and are only suitable for solving specific problems with special structures or properties. Moreover, in dynamic and fast changing vehicular network environments, the performance of optimization-based approaches often cannot be guaranteed. To address these challenges, *deep reinforcement learning (DRL [25])* has recently been introduced to solve the task offloading problem in MEC-enabled VENs [26], [27], [28], [29], [30]. These existing works mainly focused on the task offloading problem in a *single time slot*, or broke down a long-time task offloading problem into multiple small *single-slot* offloading problems. In practice, however, AI-enabled applications are often complex and cannot be completed in a single time slot. Additionally, they often cannot be easily divided into small subtasks that can be completed independently within one time slot.

In this work, we will study the cloud–edge–end collaborative offloading for *multislot* tasks in MEC-enabled VENs, where each task may require multiple time slots for processing or transmission. In such a scenario, tasks scheduled to the same computing or transmission resource block will form a *task flow*, with each task occupying the resource sequentially. To characterize the task flows on each transmission links and computing server, we propose a novel queue-based collaborative offloading framework. This framework incorporates an advanced task prioritization mechanism based on the first-in-first-out (FIFO) queue system, tailored to optimize task processing efficiency in an MEC-enabled VEN.

B. Model Scenario

As shown in Fig. 1, we consider a general MEC-enabled VEN model, which consists of multiple vehicles, multiple RSUs (each equipped with an edge computing server), and one MBS connected to the cloud computing server. By leveraging cloud–edge–end collaborative offloading, each vehicle can offload tasks to edge or cloud computing servers via V2I links or to other vehicles via V2V links. In this example, V1 processes tasks locally, V2–V4 offloads tasks to nearby vehicle via V2V links, edge computing server on RSU via V2I links, and cloud computing server through MBS via V2I links, respectively.

To characterize different task flows on each transmission link and computing server, we introduce the FIFO queue system and propose a *queue-based* cloud–edge–end collaborative offloading framework, as illustrated in Fig. 2. Specifically, each vehicle is associated with four FIFO queues: 1) *high-priority computation queue (HPC queue)*, characterizing the computation flow composed of all locally processed tasks; 2) *low-priority computation queue (LPC queue)*, characterizing the computation flow composed of all tasks offloaded from other vehicles; 3) *V2I transmission queue (V2I-T queue)*, characterizing the transmission flow composed of all tasks to be offloaded to edge or cloud computing server via V2I links; and 4) *V2V transmission queue (V2V-T queue)*, characterizing the transmission flow composed of all tasks to be offloaded to other vehicles via V2V links. Moreover, each edge infrastructure node (i.e., RSU or MBS) is associated with one FIFO queue, namely, *edge node computation queue (ENC queue)*, characterizing the computation flow composed of all tasks offloaded to the corresponding computing server.

Fig. 2 also illustrates the key workflows in such a queue-based offloading framework. For example, if a vehicle chooses to process a task locally, the task will be pushed into the HPC queue (waiting for local processing); if a vehicle chooses to offload a task to an edge or cloud computing server (on RSUs or MBS), the task will be pushed into the V2I-T queue (waiting for transmitting), and then be transmitted through V2I links into the ENC queue of the target computing server (waiting for processing); if a vehicle chooses to offload a task to another vehicle, the task will be pushed into the V2V-T queue (waiting for transmitting), and then be transmitted through V2V links into the LPC queue of the target vehicle (waiting for processing).

C. Solution and Contributions

In such a collaborative offloading framework, we are interested in the joint task offloading, scheduling, and resource

Authorized licensed use limited to: Eotvos Lorand University Budapest. Downloaded on October 27,2026 at 13:00:42 UTC from IEEE Xplore. Restrictions apply.

allocation problem for vehicles, which consists of the following problems.

1) *Offloading Problem:* Whether to offload each task?
2) *Scheduling Problem:* Where and how to offload each task, together with the V2V or V2I transmission power?
3) *Resource Allocation Problem:* How much computing capacity to be allocated for each task?

The problems are challenging due to the online and asynchronous decision-making requirement for each task. Specifically, the offloading decision is made when each task arrives, while the scheduling decision and resource allocation decision are made when each task is dequeued from the corresponding queue (indicating that all prior tasks in the queue have been completed). Such an *asynchronous* offloading, scheduling, and resource allocation decision making brings additional challenges for the joint optimization of them. Moreover, due to the fast and randomly changing of wireless environments, it is difficult to obtain complete information for offline decision making. Thus, vehicles need to make decisions in an online and distributed manner.

To deal with these challenges, we proposed a *multilayer* DRL-based framework, where each vehicle constructs and trains three modules to make different layers' decisions, as shown in Fig. 2: 1) *Offloading Module* (first layer), determining whether to offload the task when a task arrives; 2) *Scheduling Module* (second layer), determining where and how to offload the task in the V2I-T and V2V-T queues, together with the transmission power, when a task is dequeued from the corresponding queue; and 3) *Computing Module* (third layer), determining how much computing resource to be allocated for the task in the HPC and LPC queues, when a task is dequeued from the corresponding queue.

The key idea of this multilayer framework is to decouple task offloading, scheduling, and computing decisions by making decisions at different layers at different time points. This approach allows each layer's decision-making processing to dynamically adjust based on the current network state and resource availability, thereby improving the overall efficiency and responsiveness of the system. More specifically, due to the inherent complexity of binary offloading decisions, we design the offloading module to use one neural network architecture structured around the dueling and double deep Q-network (D3QN) framework. Due to the complexities of hybrid discrete scheduling and continuous transmit power decisions, we design the scheduling module to use another neural network architecture structured around the parameterized deep Q-network (PDQN) framework. Finally, we formulate the computing resource allocation problem as a convex optimization problem, and hence derive the closed-form expression of the optimal solution using classic optimization techniques through the computing module. We provide the detailed algorithm design and perform extensive simulations to evaluate its performance.

Overall, this work provides a novel framework and algorithm for task offloading in vehicular networks, addressing the challenges of online decision making and resource allocation in a dynamic and uncertain environment. In summary, the key contributions of this work are summarized as follows.

1) *Novel Model Scenario:* We consider a novel queue-based cloud–edge–end collaborative offloading framework in an MEC-enabled VEN, which incorporates an advanced task prioritization mechanism, and can significantly enhance the task offloading performance.
2) *Novel Multilayer DRL-Based Solution:* We focus on the joint task offloading, scheduling, and resource allocation problem, aiming to minimize both task latency and energy consumption of vehicles. To tackle the online and asynchronous decision making and time-varying environment, we propose a multilayer DRL-based algorithm, which enables vehicles to make optimal decisions in an online and distributed manner.
3) *Performance Evaluations:* We perform extensive simulations to evaluate its performance. Simulation results show that our proposed algorithm outperforms the existing algorithms in the literature, and can reduce the average cost by 25.86%–72.51% and increase the average satisfaction rate by 3.48%–90.53%. This highlights the algorithm's ability to provide a more cost-effective and user-friendly solution for managing computation tasks in vehicular networks.

The remainder of this work is organized as follows. In Section II, we provide the literature review. In Section III, we present the system model. In Section IV, we formulate the joint optimization problem. In Section V, we provide the multilayer DRL-based solution. We provide simulation results in Section VI and finally conclude in Section VII.

II. Literature Review

A. Cloud–Edge–End Collaborative Offloading Methods

With the rapid advancement of vehicular communication technologies, MEC has become widely used in vehicular networks for AI-enabled applications, with the goal of reducing task latency and improving the QoS. Researchers have made several noteworthy contributions in MEC-enabled VENs. For example, Bozorgchenani et al. [9] studied a task offloading problem of end-vehicles to deal with BS selection in a heterogenous scenario, while considered the varying traffic loads across different networks. Wang et al. [10] considered a VEN with dynamic topologies, unstable connections, and unpredictable movements, where end-vehicles can offload computation tasks to available neighboring vehicular edge computing clusters formed by onboard resources. Miao et al. [11] developed an original analysis model to investigate the performance of a VEN system with bursty task arrivals.

In MEC-enabled VENs, the task offloading issue is a critical and significant problem. Some existing researches in this field have explored the edge/cloud offloading mode extensively. For example, Alam and Jamalipour [13] provided a cooperative three-layer vehicle-assisted multiaccess edge network, where tasks are either processed locally at the bottom fog layer, or offloaded to edge computing servers at the middle cloudlet layer (one cloud computing server at the top cloud layer) via V2I transmission links. Dai et al. [14] integrated offloading problem with load balancing to obtain optimal resource

Authorized licensed use limited to: Eotvos Lorand University Budapest. Downloaded on October 27,2026 at 13:00:42 UTC from IEEE Xplore. Restrictions apply.

allocation for a multiuser and multiserver vehicular edge computing system. Tan et al. [15] developed a decentralized approach for task offloading and resource allocation in a multiserver VEN, where tasks can be offloaded to a RSU via V2I transmission links. Cho et al. [16] proposed a cooperative offloading model to multiple RSUs for a batch of moving vehicles by optimizing the task splitting ratio, computation resource and communication with the goal of minimizing total energy. Sun et al. [17] tackled the joint resource allocation and task offloading problem by stimulating horizontal and vertical collaboration among vehicle-edge servers or vehicle-cloud servers. Xiao et al. [18] focused on the joint optimization of security strength and resource allocation for computation offloading in vehicular edge computing systems.

The above researches have only focused on the edge/cloud offloading mode, which is unable to support multiple end-vehicles, particularly in the large scale vehicles scenario. With the increasing computational resources on end-vehicles, the end-collaborative offloading mode has emerged as another option for task offloading. For example, Sun et al. [19] studied the task offloading problem under the end-collaborative offloading mode in the vehicular edge computing systems, and proposed an adaptive learning-based offloading algorithm to minimize the average task latency.

To integrate both V2I-enabled edge/cloud offloading and V2V-enabled end collaboration, it is worth exploring a cloud–edge–end collaborative offloading mode. To study the cloud–edge–end collaborative task offloading problem, a variety of optimization-based approaches have been proposed, including Lyapunov optimization [21] and particle swarm optimization (PSO) [22], which are shown promising results. Game theory [23], [24] is used to incentivize vehicles to share computing resources. For example, Yue and Wang [21] presented a Lyapunov-based dynamic computation offloading algorithm, which transforms the formulated problem into a series of subproblems by minimizing the derived upper bound of the Lyapunov drift-plus-penalty function. Hou et al. [22] designed a fault-tolerant PSO algorithm for maximizing the reliability with latency constraints. Wang et al. [23] constructed a task offloading decision-based game to minimize the computation overhead. Pham et al. [24] adopted game-theoretic approach to devise a low-complexity distributed offloading scheme, where the offloading problem is modeled as an exact potential game and resource allocation problem are determined by subgradient method. However, these optimization-based approaches only are used to solve special problems with specific structures or properties. Moreover, the performance of these approaches often cannot be guaranteed due to the fast changing of the environment.

B. DRL-Based Offloading Methods

DRL is a promising approach to make online decisions under fast-changing and uncertain environments to address these challenges. Several existing researches (e.g. [27], [28], [29], and [30]) have utilized a model-free approach with DRL to solve task offloading or resource allocation problems in MEC-enabled VENs. For example, Shi et al. [27] proposed a DRL-based computation offloading approach to guarantee the reliability of resource allocation in blockchain-enabled vehicular networks. Liang et al. [28] proposed a multiagent deep Q-network (DQN) approach for solving the multiobjective spectrum sharing problem during task offloading in MEC-enabled VENs. Luo et al. [29] proposed a DRL-based approach to solve the collaborative data scheduling problem in MEC-enabled VENs. Zhang et al. [30] designed a DRL-based transmission mode selection and resource allocation algorithm for MEC-enabled VENs, aiming at maximizing the sum capacity of all users while guaranteeing the latency and reliability requirements. The above works mainly focused on the task offloading problem in a single time slot without considering the multislot scenario. Furthermore, they did not characterize the queuing system of multiple tasks applying for the same resource. Additionally, these works mainly considered edge/cloud offloading mode, and few of them considered a cloud–edge–end collaborative offloading framework where each task can be offloaded to end-vehicles, edge servers, and cloud servers.

TABLE I
COMPARISON OF OUR WORK WITH EXISTING WORKS

	V2I	V2V	Decision Mode	Solution Technique	Method
[13]	Y	N	Online	Learning	DQN
[14]–[16]	Y	N	Offline	Optimization	KKT
[17]	Y	N	Online	Optimization	Game
[18]	Y	N	Offline	Optimization	GBD
[19]	N	Y	Online	Learning	MAB
[20]	Y	N	Offline	Optimization	Lyapunov
[21]	Y	Y	Offline	Optimization	Lyapunov
[22]	Y	Y	Offline	Optimization	PSO
[23], [24]	Y	Y	Offline	Optimization	Game
[26]	Y	N	Online	Learning	DQN
[27]	N	Y	Online	Learning	DQN
[28]–[30]	Y	Y	Online	Learning	DQN
our	Y	Y	Online	Learning	DQN,PDQN

Based on the above discussion, our focus will be on the cloud–edge–end collaborative offloading framework for multislot tasks in MEC-enabled VENs, where each task may require multiple time slots for processing or transmission. Specifically, we utilize the FIFO queue systems to manage task flows composed of tasks scheduled to the same computing or transmission resource block. In such a scenario, we will explore the joint task offloading, scheduling, and resource allocation problem for vehicles. To solve this problem, we will propose a multilayer DRL-based approach that facilitates asynchronous decision making, ultimately enhancing the overall performance of MEC-enabled VENs. Table I highlights the contribution of our work compared with existing works using different methods, such as Karush–Kuhn–Tucker (KKT), multiarmed bandit (MAB), and generalized benders decomposition (GBD).

III. SYSTEM MODEL

A. Network Model

As shown in Fig. 1, we consider a general MEC-enabled VEN model[1] consisting of a set $\mathcal{I} \triangleq \{1, 2, \ldots, I\}$ of I vehicles,

[1]This model is a linear road segment network, which accurately simulates typical road sections found in real-world traffic environments.

Authorized licensed use limited to: Eotvos Lorand University Budapest. Downloaded on October 27,2026 at 13:00:42 UTC from IEEE Xplore. Restrictions apply.

TABLE II
MAJOR NOTATIONS

Notation	Explanation
$\mathcal{I}$, I	Set and number of vehicles
$\mathcal{J}$, J	Set and number of edge nodes
$\mathcal{T}$, T	Set and number of time slots
$w_{i,k}$	The k-th task generated by vehicle i
$d_{i,k}$, $c_{i,k}$	Data size, the number of CPU cycles
$\tau_{i,k}$, $\delta_{i,k}$	Generation time, deadline
$\boldsymbol{x}_{i,k}$	Offloading decision
$\mathcal{M}_j$, M_j	Set and number of sub-channels by edge node j
$\boldsymbol{y}_{i,k}$, $p_{i,k}^{\mathrm{V2I}}$	V2I scheduling/V2I transmit power decision
$\boldsymbol{z}_{i,k}$, $p_{i,k}^{\mathrm{V2V}}$	V2V scheduling/V2V transmit power decision
$f_{i,k}^{\mathrm{HPC}}$, $f_{i',k',i}^{\mathrm{LPC}}$	High/Low-priority computing capacity decision
f_j^{EN}	The computing capacity of edge node j
$T_{i,k}^{\mathrm{locw}}$, $T_{i,k}^{\mathrm{locc}}$	Local waiting time, local computation time
Q_i^{HPC}	Ordered set of tasks in the HPC queue
$T_{i,k}^{\mathrm{V2Iw}}$, $T_{i,k}^{\mathrm{V2It}}$	V2I waiting/transmission time
$T_{i,k}^{\mathrm{ENw}}$, $T_{i,k}^{\mathrm{ENc}}$	Edge node waiting/computation time
Q_i^{V2I}	Ordered set of tasks in the V2I queue
$T_{i,k}^{\mathrm{V2Vw}}$, $T_{i,k}^{\mathrm{V2Vt}}$	V2V waiting/transmission time
$T_{i,k}^{\mathrm{Vw}}$, $T_{i,k}^{\mathrm{Vc}}$	Vehicle waiting/computation time
Q_i^{V2V}	Ordered set of tasks in the V2V queue

and a set $\mathcal{J} \triangleq \{1, 2, \ldots, J, J+1\}$ of $J+1$ edge nodes (including J RSUs and one MBS), where $j \leq J$ denotes a particular RSU and $j = J+1$ denotes the MBS. Each vehicle with limited computing resources generates dynamically computation tasks with the Poisson distribution, while moves with a constant speed v_i, $i \in \mathcal{I}$. Each RSU is equipped with an edge server with limited computing resources, and the MBS is connected to the cloud server with abundant computing resources. Vehicles can offload their tasks to RSUs (edge computing servers) and the MBS (cloud computing server) via V2I links, or to other vehicles via V2V links. For ease of analysis, we adopt a *time-slotted* structure, where each time period (called an episode) is divided into a set $\mathcal{T} \triangleq \{1, 2, \ldots, T\}$ of T time slots, each with an equal length of Δ. As in many existing literatures (e.g., [31], [32], [33], and [34]), we consider a quasi-static system, where the network state stays unchanged within each time slot, but may randomly change across the different time slots. For convenience, the major notations are summarized in Table II.

B. Vehicle Model

In the following, we first present the task model and the queue model, and then analyze the behaviors (i.e., decision variables) of each vehicle in details.

1) Task Model: At the beginning of time slot $t \in \mathcal{T}$, each vehicle generates new tasks following the Poisson distribution [21], [22], [23] with the expectation of λ, i.e., it may generate multiple tasks or not generate any task. Let K_i and $\mathcal{K}_i$ denote the total number of tasks generated by vehicle i and the newly arrival tasks set of vehicle i at one episode, respectively. That is, $\mathcal{K}_i \triangleq \{1, 2, \ldots, K_i\}$. For ease of writing, let $\mathcal{K}_i(t)$ denote the task set generated by vehicle i at time slot t, and then we have $\mathcal{K}_i = \cup_{t \in \mathcal{T}} \mathcal{K}_i(t)$. To characterize the attribution of tasks, we introduce a notation

$$w_{i,k} \triangleq \left(d_{i,k}, c_{i,k}, \tau_{i,k}, \delta_{i,k}\right), \quad k \in \mathcal{K}_i(t)$$

to represent the unique index of the kth task generated by vehicle i. The term $d_{i,k} \in [d_i^{\min}, d_i^{\max}]$ denotes the data size (in bits) of the task, where $d_i^{\min}$ and $d_i^{\max}$ are the minimum and maximum data sizes. The term $c_{i,k} \in [c_i^{\min}, c_i^{\max}]$ denotes the number of CPU cycles required for processing the task, where $c_i^{\min}$ and $c_i^{\max}$ are the number of the minimum and maximum CPU cycles. The term $\tau_{i,k} \in \mathcal{T}$ denotes the generation time of the task. The term $\delta_{i,k} \in [\delta_i^{\min}, \delta_i^{\max}]$ denotes a deadline (in seconds) of the task, where $\delta_i^{\min}$ and $\delta_i^{\max}$ are the minimum and maximum deadline.

2) Queue Model: As shown in Fig. 2, when a new task is generated by one vehicle, the offloading module of the vehicle decides whether to process the task locally, or offload the task to a target MEC server or the cloud server via V2I links, or offload the task to a target vehicle via V2V links, or drop the task directly. To characterize different task flows on each transmission link and computing server, we introduce a HPC queue and a LPC queue for each vehicle to record the sets of tasks to be processed locally, while introduce a V2I-T queue and a V2V-T queue for each vehicle to record the sets of tasks to be transmitted. All of the above queues follow the FIFO principle, and tasks in the queues will be processed or transmitted sequentially. Besides, tasks in the HPC queue are processed with high-priority computing capacity, while tasks in the LPC queue are processed with low-priority computing capacity.

Next, we illustrate the details of these queues for vehicles. When a task of one vehicle is pushed into the HPC queue (or the LPC queue) of the vehicle, it will wait in the queue (for local processing) until all prior tasks in the HPC queue (or the LPC queue) are completed, and then the computing module of the vehicle will assign the computing capacity for processing the task. When a task of one vehicle is pushed into the V2I-T queue (or the V2V-T queue) of the vehicle, it will wait in the queue (for offloading) until all prior tasks in the V2I-T queue (or the V2V-T queue) are fully transmitted, and then the scheduling module of the vehicle will select a target server (or a target vehicle) and assign the transmit power for offloading the task via V2I links (or via V2V links). Note that if all of the above ways may not meet the task's latency requirement, the vehicle can also drop the task with a certain penalty.

3) Decision Variables: As shown in Fig. 2, when a task $w_{i,k}$ of vehicle i arrives at time slot t (i.e., generation time $\tau_{i,k}$), the offloading module of the vehicle decides whether to process the task locally, or offload the task to a target MEC server or the cloud server via V2I links, or offload the task to a target vehicle via V2V links, or drop the task directly. Thus, we introduce a notation

$$\boldsymbol{x}_{i,k} \triangleq \left(x_{i,k}^{\mathrm{dro}}, x_{i,k}^{\mathrm{loc}}, x_{i,k}^{\mathrm{V2I}}, x_{i,k}^{\mathrm{V2V}}\right)$$

to represent such a *task offloading* decision, where $x_{i,k}^{\mathrm{dro}} \in \{0, 1\}$ denotes whether the task is dropped; $x_{i,k}^{\mathrm{loc}} \in \{0, 1\}$ denotes whether the task is processed locally; $x_{i,k}^{\mathrm{V2I}} \in \{0, 1\}$ denotes whether the task is offloaded via V2I links; and $x_{i,k}^{\mathrm{V2V}} \in \{0, 1\}$ denotes whether the task is offloaded via V2V links. Note that

Authorized licensed use limited to: Eotvos Lorand University Budapest. Downloaded on October 27,2026 at 13:00:42 UTC from IEEE Xplore. Restrictions apply.

each task must be chosen in one place, i.e.,

$$\sum_{n\in\Omega} x_{i,k}^{n} = 1 \ \forall i \in \mathcal{I}, k \in \mathcal{K}_i(t), t \in \mathcal{T} \tag{1}$$

where $\Omega \triangleq \{\text{dro, loc, V2I, V2V}\}$.

As in many existing literatures (e.g., [35] and [36]), we assume that edge nodes (RSUs or MBS) adopt the orthogonal frequency division multiplexing (OFDM) technique for V2I transmission. Let $\mathcal{M}_j \triangleq \{1, 2, \ldots, M_j\}$ denote the set of OFDM subchannels used by edge node $j \in \mathcal{J}$, where M_j is the number of subchannels at edge node j. To facilitate analysis and implementation, we further assume that the subchannels used by different RSUs or MBS do not interfere with each other. When a task $w_{i,k}$ of vehicle i is pushed into the V2I-T queue of the vehicle, it will wait in the queue until all previous tasks in the V2I-T queue are fully transmitted (say, at time t_1^{V2I} where $t_1^{\text{V2I}} \geq \tau_{i,k}$), and then the scheduling module of the vehicle will make V2I scheduling and V2I transmit power decisions for sending the task via V2I links. Thus, we introduce two notations

$$\boldsymbol{y}_{i,k} \triangleq \{y_{i,k,m,j} | j \in \mathcal{J}, m \in \mathcal{M}_j\} \text{ and } p_{i,k}^{\text{V2I}} \in \left[0, p_i^{\text{V2Imax}}\right]$$

to represent such *V2I scheduling and V2I transmit power decisions*, where $y_{i,k,m,j} \in \{0, 1\}$ denotes whether to offload the task $w_{i,k}$ to edge node j through channel m at time t_1^{V2I} and p_i^{V2Imax} is the maximum V2I transmit power. Note that each task in the V2I-T queue must be transmitted to one edge node for edge processing, i.e.,

$$\sum_{j\in\mathcal{J}} \sum_{m\in\mathcal{M}_j} y_{i,k,m,j} = x_{i,k}^{\text{V2I}} \ \forall i \in \mathcal{I}, k \in \mathcal{K}_i(t), t \in \mathcal{T}. \tag{2}$$

And one subchannel can only be allocated to one vehicle at a certain time, i.e.,

$$\sum_{i\in\mathcal{I}} y_{i,k,m,j} \leq 1 \ \forall k \in \mathcal{K}_i(t), t \in \mathcal{T}, j \in \mathcal{J}, m \in \mathcal{M}_j. \tag{3}$$

It is important to note that such V2I scheduling and V2I transmit power decisions for task $w_{i,k}$ may be not made at the generation time $\tau_{i,k}$ immediately.

Similarly, when a task $w_{i,k}$ of vehicle i is pushed into the V2V-T queue of the vehicle, it will wait in the queue until all previous tasks in the V2V-T queue are fully transmitted (say, at time t_1^{V2V} where $t_1^{\text{V2V}} \geq \tau_{i,k}$), and then the scheduling module of the vehicle will make V2V scheduling and V2V transmit power decisions for sending the task via V2V links. Thus, we introduce two notations

$$\boldsymbol{z}_{i,k} \triangleq \{z_{i,k,i'} | i' \in \mathcal{I} \setminus \{i\}\} \text{ and } p_{i,k}^{\text{V2V}} \in \left[0, p_i^{\text{V2Vmax}}\right]$$

to represent such *V2V scheduling and V2V transmit power decisions*, where $z_{i,k,i'} \in \{0, 1\}$ denotes whether to offload the task $w_{i,k}$ to vehicle i' at time t_1^{V2V} and p_i^{V2Vmax} is the maximum V2V transmit power. Note that one vehicle can only connect with another vehicle via V2V links at a certain time, i.e.,

$$\sum_{i'\in\mathcal{I}\setminus\{i\}} z_{i,k,i'} = x_{i,k}^{\text{V2V}} \ \forall i \in \mathcal{I}, k \in \mathcal{K}_i(t), t \in \mathcal{T} \tag{4}$$

$$\sum_{i\in\mathcal{I}} z_{i,k,i'} \leq 1 \ \forall k \in \mathcal{K}_i(t), t \in \mathcal{T}, i' \in \mathcal{I} \setminus \{i\}. \tag{5}$$

It is importance to note that such V2V scheduling and V2V transmit power decisions for task $w_{i,k}$ may be not made at the generation time $\tau_{i,k}$ immediately.

More specifically, when a task $w_{i,k}$ of vehicle i is pushed into the HPC queue of the vehicle, it will wait in the queue (for local processing) until all previous tasks in the HPC queue are completed (say, at time t_1^{loc} where $t_1^{\text{loc}} \geq \tau_{i,k}$), and then the computing module of the vehicle will assign the high-priority computing capacity decision $f_{i,k}^{\text{HPC}} \in [0, \eta_1 f_i^{\max}]$ for processing the task, where $f_i^{\max}$ is the maximum computing capacity of vehicle i and η_1 is the ratio of computing capacity allocated by the vehicle. Similarly, when a task $w_{i',k'}$ of vehicle i' is pushed into the LPC queue of vehicle i, it will wait in the queue (for vehicle processing) until all previous tasks in the LPC queue are completed, and then the computing module of vehicle i will assign the low-priority computing capacity decision $f_{i',k',i}^{\text{LPC}} \in [0, \eta_2 f_i^{\max}]$, where $\eta_2 (< \eta_1)$ is the ratio of computing capacity allocated by vehicle i.

C. Edge Node Model

As shown in Fig. 2, to effectively manage and characterize different task flows on computing server, we introduce an ENC queue for each edge node (RSUs or MBS). This ENC queue records the sets of offloaded tasks from vehicles that need to be processed on the computing server of the corresponding edge node. All ENC queues follow the FIFO principle, and tasks in the queues will be processed sequentially. For ease of the following description, let f_j^{EN} denote the computing capacity of edge node $j \in \mathcal{J}$.

D. Communication Model

As shown in Fig. 2, tasks in the V2I-T queue and the V2V-T queue need to be scheduled to a target edge node and a target vehicle via V2I links and V2V links. Thus, we provide the details of the V2I links and the V2V links in the following.

1) V2I Links: If a task of one vehicle in the V2I-T queue is ready for data transmission, the scheduling module of the vehicle needs to immediately choose a target edge node, together with channel selection and transmit power, for sending the task via V2I links. Due to the multiple time slots of sending the task, we denote the average transmission rate for task $w_{i,k}$ between vehicle $i \in \mathcal{I}$ and edge node $j \in \mathcal{J}$ through channel $m \in \mathcal{M}_j$ as

$$\tilde{r}_{i,k,m,j} \triangleq \frac{1}{t_2 - t_1} \int_{t_1}^{t_2} W_j \log_2\left(1 + \frac{|H_{i,m,j}(t)|^2 p_{i,k}^{\text{V2I}}}{\sigma^2}\right) dt$$

where t_1 and t_2 are the start and end times of sending the task; $H_{i,m,j}(t)$ is the channel gain between vehicle i and edge node j on channel m at time t; and σ^2 and W_j are the variance of the additive white Gaussian noise and the allocated bandwidth. Thus, the achieved V2I transmission rate of vehicle i for task $w_{i,k}$ can be calculated in the following way:

$$r_{i,k}^{\text{V2I}} = \sum_{j\in\mathcal{J}} \sum_{m\in\mathcal{M}_j} y_{i,k,m,j} \tilde{r}_{i,k,m,j}. \tag{6}$$

Authorized licensed use limited to: Eotvos Lorand University Budapest. Downloaded on October 27,2026 at 13:00:42 UTC from IEEE Xplore. Restrictions apply.

2) V2V Links: As in existing literature [37], we assume that vehicles adopt the D2D technique (e.g., WiFi direct) for V2V transmission. Similarly, if a task of one vehicle in the V2V-T queue is ready for data transmission, the scheduling module of the vehicle needs to immediately choose a target vehicle, together with the transmit power, for sending the task via V2V links. Due to the multiple time slots of sending the task, we denote the average transmission rate for task $w_{i,k}$ between vehicle i and vehicle i' as

$$\tilde{r}_{i,k,i'} \triangleq \frac{1}{t_4 - t_3} \int_{t_3}^{t_4} W_i \log_2\left(1 + \frac{|H_{i,i'}(t)|^2 p_{i,k}^{\mathrm{V2V}}}{\sigma^2 + \Theta}\right) dt$$

where t_3 and t_4 are the start and end times of sending the task; $H_{i,i'}(t)$ is the channel gain between vehicle i and vehicle i' at time t; and W_i is the allocated bandwidth; Θ is the interference from other vehicles. Thus, the achieved V2V transmission rate of vehicle i for task $w_{i,k}$ can be calculated in the following way:

$$r_{i,k}^{\mathrm{V2V}} = \sum_{i' \in \mathcal{I} \setminus \{i\}} z_{i,k,i'} \tilde{r}_{i,k,i'}. \tag{7}$$

E. Queue-Based Cloud–Edge–End Collaborative Offloading Framework

Based on the discussed models, we will illustrate the details of local, V2I, V2V, and drop ways in the following.

1) Local Way ($x_{i,k}^{loc} = 1$): As shown in Fig. 2, if vehicle i chooses to process its task $w_{i,k}$ locally, the task is pushed into the HPC queue of the vehicle for local processing. Thus, the total latency $T_{i,k}^{\mathrm{loc}}$ for the task contains the local waiting time $T_{i,k}^{\mathrm{locw}}$ and the local computation time $T_{t,i}^{\mathrm{locc}}$, i.e.,

$$T_{i,k}^{\mathrm{loc}} = T_{i,k}^{\mathrm{locw}} + T_{i,k}^{\mathrm{locc}}. \tag{8}$$

When a task $w_{i,k}$ of vehicle i is pushed into the HPC queue of the vehicle at time slot $\tau_{i,k}$, it will observe an ordered task set $Q_i^{\mathrm{HPC}}(\tau_{i,k})$[2] for the task, whose element $k' \in Q_i^{\mathrm{HPC}}(\tau_{i,k})$ represents task $w_{i,k'}$ has been pushed into the HPC queue before the current time slot $\tau_{i,k} (\geq \tau_{i,k'})$. The task need to wait for local processing until all previous tasks in the HPC queue have been fully completed. It is important to note that a specific task $w_{i,\hat{k}}^{\mathrm{HPC}}$ may not be completed. We denote $\phi_{i,\hat{k}}^{\mathrm{HPC}} \in [0, 1]$ as the percentage of remaining uncompleted workload of the $\hat{k}$th task in the queue $Q_i^{\mathrm{HPC}}(\tau_{i,k})$. Thus, the local waiting time $T_{i,k}^{\mathrm{locw}}$ of task $w_{i,k}$ can be calculated in the following:

$$T_{i,k}^{\mathrm{locw}} = \sum_{k' \in Q_i^{\mathrm{HPC}}(\tau_{i,k})} \frac{c_{i,k'}}{f_{i,k'}^{\mathrm{HPC}}} + \frac{\phi_{i,\hat{k}}^{\mathrm{HPC}} c_{i,\hat{k}}^{\mathrm{HPC}}}{f_{i,\hat{k}}^{\mathrm{HPC}}}.$$

At time $t_1^{\mathrm{loc}} = \tau_{i,k} + T_{i,k}^{\mathrm{locw}}$ (i.e., the time at which the task is ready for processing on the vehicle), the computing module of the vehicle will assign a computing capacity decision $f_{i,k}^{\mathrm{HPC}}$ for task $w_{i,k}$. Thus, the local computation time $T_{i,k}^{\mathrm{locc}}$ and the local energy consumption $E_{i,k}^{\mathrm{loc}}$ of task $w_{i,k}$ can be calculated in the following:

$$T_{i,k}^{\mathrm{locc}} = \frac{c_{i,k}}{f_{i,k}^{\mathrm{HPC}}}, \quad E_{i,k}^{\mathrm{loc}} = \xi_i \left(f_{i,k}^{\mathrm{HPC}}\right)^2 c_{i,k}$$

where ξ_i is the energy efficient coefficient of vehicle i. Note that the energy consumption for local way includes the energy consumption required for completing tasks directly on the vehicle.

2) V2I Way ($x_{i,k}^{V2I} = 1$): As shown in Fig. 2, if vehicle i chooses to offload its task $w_{i,k}$ to a target edge node (RSUs or MBS) via V2I links, the task is pushed into the V2I-T queue of the vehicle for data transmission. Furthermore, if the task in the V2I-T queue is scheduled to an edge node j in the future, it will be pushed into the ENC queue of edge node j and wait for edge processing. Thus, the total latency $T_{i,k}^{\mathrm{V2I}}$ for the task contains the V2I waiting time $T_{i,k}^{\mathrm{V2Iw}}$, the V2I transmission time $T_{i,k}^{\mathrm{V2It}}$, the edge node waiting time $T_{i,k}^{\mathrm{ENw}}$, and the edge node computation time $T_{i,k}^{\mathrm{ENc}}$, i.e.,

$$T_{i,k}^{\mathrm{V2I}} = T_{i,k}^{\mathrm{V2Iw}} + T_{i,k}^{\mathrm{V2It}} + T_{i,k}^{\mathrm{ENw}} + T_{i,k}^{\mathrm{ENc}}. \tag{9}$$

When a task $w_{i,k}$ of vehicle i is pushed into the V2I-T queue of the vehicle at time slot $\tau_{i,k}$, it will observe an ordered task set $Q_i^{\mathrm{V2I}}(\tau_{i,k})$ for the task. The task need to wait for data transmission until all previous tasks in the V2I-T queue have been fully transmitted. It is important to note that a specific task $w_{i,\hat{k}}^{\mathrm{V2I}}$ may not be fully transmitted. We denote $\phi_{i,\hat{k}}^{\mathrm{V2I}} \in [0, 1]$ as the percentage of remaining untransmitted workload of the $\hat{k}$th task in the queue $Q_i^{\mathrm{V2I}}(\tau_{i,k})$. Thus, the V2I waiting time $T_{i,k}^{\mathrm{V2Iw}}$ of task $w_{i,k}$ can be calculated in the following:

$$T_{i,k}^{\mathrm{V2Iw}} = \sum_{k' \in Q_i^{\mathrm{V2I}}(\tau_{i,k})} \frac{d_{i,k'}}{r_{i,k'}^{\mathrm{V2I}}} + \frac{\phi_{i,\hat{k}}^{\mathrm{V2I}} d_{i,\hat{k}}^{\mathrm{V2I}}}{r_{i,\hat{k}}^{\mathrm{V2I}}}.$$

At time $t_1^{\mathrm{V2I}} = \tau_{i,k} + T_{i,k}^{\mathrm{V2Iw}}$ (i.e., the time at which the task is ready for transmitting), the scheduling module of the vehicle will output the V2I scheduling decision $\boldsymbol{y}_{i,k}$ and the V2I transmit power $p_{i,k}^{\mathrm{V2I}}$ for task $w_{i,k}$. Thus, the V2I transmission time $T_{i,k}^{\mathrm{V2It}}$ of task $w_{i,k}$ can be calculated in the following:

$$T_{i,k}^{\mathrm{V2It}} = \frac{d_{i,k}}{r_{i,k}^{\mathrm{V2I}}}.$$

At time $t_2^{\mathrm{V2I}} = t_1^{\mathrm{V2I}} + T_{i,k}^{\mathrm{V2It}}$ (i.e., the time at which the task will arrive at the ENC queue of edge node $j \in \mathcal{J}$), it will observe an ordered task set $Q_j^{\mathrm{ENC}}(t_2^{\mathrm{V2I}})$[3] for the task, whose element $(i', k') \in Q_j^{\mathrm{ENC}}(t_2^{\mathrm{V2I}})$ represents task $w_{i',k'}$ has been pushed into the ENC queue of edge node j before the time $t_2^{\mathrm{V2I}} (\geq \tau_{i',k'})$. It is important to note that a specific task $w_{\hat{i},\hat{k}}$ may not be fully completed. We denote $\phi_{\hat{i},\hat{k}}^{\mathrm{EN}} \in [0, 1]$ as the percentage of remaining uncompleted workload of the $\hat{k}$th task of vehicle $\hat{i}$ in the queue $Q_j^{\mathrm{ENC}}(t_2^{\mathrm{V2I}})$. Thus, the edge node

[2]Without loss of generality, we can denote $Q_i^{\mathrm{HPC}}(\tau_{i,k}) = \{k_1, k_2, \ldots, k_{N_{i,k}}\}$ at time $\tau_{i,k}$, where $N_{i,k}$ is the number of tasks in the set. Note that when a task has been finished, a new task can be taken from the set for local processing.

[3]We denote $Q_j^{\mathrm{ENC}}(t_2^{\mathrm{V2I}}) = \{(i_1, k_1), (i_2, k_2), \ldots, (i_{\hat{N}_{i,k}}, k_{\hat{N}_{i,k}})\}$ at time $t_2^{\mathrm{V2I}} = t_1^{\mathrm{V2I}} + T_{i,k}^{\mathrm{V2It}}$, where $\hat{N}_{i,k}$ is the number of tasks in the set.

Authorized licensed use limited to: Eotvos Lorand University Budapest. Downloaded on October 27,2026 at 13:00:42 UTC from IEEE Xplore. Restrictions apply.

waiting time $T_{i,k}^{\mathrm{ENw}}$ and the edge node computation time $T_{i,k}^{\mathrm{ENc}}$ of task $w_{i,k}$ can be calculated in the following:

$$T_{i,k}^{\mathrm{ENw}} = \sum_{j\in\mathcal{J}}\sum_{m\in\mathcal{M}_j} y_{i,k,m,j} \sum_{(i',k')\in Q_j^{\mathrm{ENC}}(t_2^{\mathrm{V2I}})} \frac{c_{i',k'}}{f_j^{\mathrm{EN}}} + \frac{\phi_{\hat{i},\hat{k}}^{\mathrm{EN}} c_{\hat{i},\hat{k}}^{\mathrm{EN}}}{f_j^{\mathrm{EN}}}$$

$$T_{i,k}^{\mathrm{ENc}} = \sum_{j\in\mathcal{J}}\sum_{m\in\mathcal{M}_j} y_{i,k,m,j} \frac{c_{i,k}}{f_j^{\mathrm{EN}}}.$$

More specifically, the energy consumption of the vehicle for V2I way can be calculated in the following:

$$E_{i,k}^{\mathrm{V2I}} = p_{i,k}^{\mathrm{V2I}} T_{i,k}^{\mathrm{V2It}}.$$

Note that the energy consumption for V2I way includes the energy consumption involved when tasks are transmitted to edge nodes for processing. As RSUs and MBS are often with rich electricity supply, we ignore their energy consumptions.

3) V2V Way ($x_{i,k}^{V2V} = 1$): As shown in Fig. 2, if vehicle i chooses to offload its task $w_{i,k}$ to a target vehicle via V2V links, the task is pushed into the V2V-T queue of the vehicle for data transmission. Furthermore, if the task in the V2V-T queue is scheduled to a vehicle i' in the future, it will be pushed into the LPC queue of vehicle i' to wait for vehicle processing. Thus, the total latency $T_{i,k}^{\mathrm{V2V}}$ for the task contains the V2V waiting time $T_{i,k}^{\mathrm{V2Vw}}$, the V2V transmission time $T_{i,k}^{\mathrm{V2Vt}}$, the vehicle waiting time $T_{i,k}^{\mathrm{Vw}}$, and the vehicle computation time $T_{i,k}^{\mathrm{Vc}}$, i.e.,

$$T_{i,k}^{\mathrm{V2V}} = T_{i,k}^{\mathrm{V2Vw}} + T_{i,k}^{\mathrm{V2Vt}} + T_{i,k}^{\mathrm{Vw}} + T_{i,k}^{\mathrm{Vc}}. \tag{10}$$

When a task $w_{i,k}$ of vehicle i is pushed into the V2V-T queue of the vehicle at time slot $\tau_{i,k}$, it will observe an ordered task set $Q_i^{\mathrm{V2V}}(\tau_{i,k})$ for the task. The task need to wait for data transmission until all previous tasks in the V2V-T queue have been fully transmitted. It is important to note that a specific task $w_{i,\hat{k}}^{\mathrm{V2V}}$ may not be fully transmitted. We denote $\phi_{i,\hat{k}}^{\mathrm{V2V}} \in [0,1]$ as the percentage of remaining untransmitted workload of the $\hat{k}$th task in the queue $Q_i^{\mathrm{V2V}}(\tau_{i,k})$. Thus, the V2V waiting time $T_{i,k}^{\mathrm{V2Vw}}$ of task $w_{i,k}$ can be calculated in the following:

$$T_{i,k}^{\mathrm{V2Vw}} = \sum_{k'\in Q_i^{\mathrm{V2V}}(\tau_{i,k})} \frac{d_{i,k'}}{r_{i,k'}^{\mathrm{V2V}}} + \frac{\phi_{i,\hat{k}}^{\mathrm{V2V}} d_{i,\hat{k}}^{\mathrm{V2V}}}{r_{i,\hat{k}}^{\mathrm{V2V}}}.$$

At time $t_1^{\mathrm{V2V}} = \tau_{i,k} + T_{i,k}^{\mathrm{V2Vw}}$ (i.e., the time at which the task is ready for transmitting), the scheduling module of the vehicle will output the V2V scheduling decision $z_{i,k}$ and the V2V transmit power $p_{i,k}^{\mathrm{V2V}}$ for task $w_{i,k}$. Thus, the V2V transmission time $T_{i,k}^{\mathrm{V2Vt}}$ can be calculated in the following:

$$T_{i,k}^{\mathrm{V2Vt}} = \frac{d_{i,k}}{r_{i,k}^{\mathrm{V2V}}}.$$

At time $t_2^{\mathrm{V2V}} = t_1^{\mathrm{V2V}} + T_{i,k}^{\mathrm{V2Vt}}$ (i.e., the time at which the task will arrive at the LPC queue of vehicle i'), it will observe an ordered task set $Q_{i'}^{\mathrm{LPC}}(t_2^{\mathrm{V2V}})$ for the task. It is important to note that a specific task $w_{\hat{i},\hat{k}}^{\mathrm{LPC}}$ may not be completed. We denote $\phi_{\hat{i},\hat{k}}^{\mathrm{LPC}} \in [0,1]$ as the percentage of remaining uncompleted workload of the $\hat{k}$th task of vehicle $\hat{i}$ in the queue $Q_{i'}^{\mathrm{LPC}}(t_2^{\mathrm{V2V}})$. Thus, the vehicle waiting time $T_{i,k}^{\mathrm{Vw}}$ and the vehicle computation time $T_{i,k}^{\mathrm{Vc}}$ of task $w_{i,k}$ can be calculated in the following:

$$T_{i,k}^{\mathrm{Vw}} = \sum_{i'\in\mathcal{I}\setminus\{i\}} z_{i,k,i'} \sum_{(i'',k'')\in Q_{i'}^{\mathrm{LPC}}(t_2^{\mathrm{V2V}})} \frac{c_{i'',k''}}{f_{i'',k'',i'}^{\mathrm{LPC}}} + \frac{\phi_{\hat{i},\hat{k}}^{\mathrm{LPC}} w_{\hat{i},\hat{k}}^{\mathrm{LPC}}}{f_{\hat{i},\hat{k},i'}^{\mathrm{LPC}}}$$

$$T_{i,k}^{\mathrm{Vc}} = \sum_{i'\in\mathcal{I}\setminus\{i\}} z_{i,k,i'} \frac{c_{i,k}}{f_{i,k,i'}^{\mathrm{LPC}}}.$$

More specifically, the energy consumption of the vehicle for V2V way can be calculated in the following:

$$E_{i,k}^{\mathrm{V2V}} = p_{i,k}^{\mathrm{V2V}} T_{i,k}^{\mathrm{V2Vt}} + \sum_{i'\in\mathcal{I}\setminus\{i\}} z_{i,k,i'} \xi_{i'} \left(f_{i,k,i'}^{\mathrm{LPC}}\right)^2 c_{i,k}$$

where the first term is the energy consumption for data transmission on vehicle i and the second term is the energy consumption for data computation on vehicle i'. Note that the energy consumption for V2V way includes the energy consumption for transmitting tasks between vehicles and required for processing these tasks on the target vehicle.

4) Drop Way ($x_{i,k}^{dro} = 1$): In this way where the total latency of a task $w_{i,k}$ exceeds its deadline $\delta_{i,k}$, an additional penalty μ will be imposed on top of the incurred energy and latency costs. Note that if the vehicle is able to predict such a latency violation, it can drop the task directly, which will incur the penalty only, i.e., without incurring the energy and latency

F. Problem Description

Based on the above analysis, we will study the joint task offloading, scheduling and resource allocation problem for vehicles, aiming at minimizing both latency and energy consumption in the long term. The problem is very challenging due to asynchronous offloading, scheduling and resource allocation decisions as well as dynamic environment. DRL approach is a widely used tool for dealing with dynamic environments and tackling asynchronous decisions, making them well-suited for this problem. Therefore, we will leverage the DRL approach to solve the joint task offloading, scheduling and resource allocation problem in this work.

IV. Problem Formulation

In this section, we will formulate the joint task offloading, scheduling, and resource allocation problem, aiming at optimizing both latency and energy consumption in the whole time period. Based on the discussion in Section III, the latency and energy consumption for task $w_{i,k}$ can be calculated in the following:

$$T_{i,k} = x_{i,k}^{\mathrm{loc}} T_{i,k}^{\mathrm{loc}} + x_{i,k}^{\mathrm{V2I}} T_{i,k}^{\mathrm{V2I}} + x_{i,k}^{\mathrm{V2V}} T_{i,k}^{\mathrm{V2V}} \tag{11}$$

$$E_{i,k} = x_{i,k}^{\mathrm{loc}} E_{i,k}^{\mathrm{loc}} + x_{i,k}^{\mathrm{V2I}} E_{i,k}^{\mathrm{V2I}} + x_{i,k}^{\mathrm{V2V}} E_{i,k}^{\mathrm{V2V}}. \tag{12}$$

Besides, if the latency of task $w_{i,k}$ exceeds its deadline $\delta_{i,k}$, there will incur additional penalty, i.e.,

$$\Phi_{i,k} = \left(1 - x_{i,k}^{\mathrm{dro}}\right) \mathbb{I}\left(T_{i,k} > \delta_{i,k}\right) \mu + x_{i,k}^{\mathrm{dro}} \mu \tag{13}$$

Authorized licensed use limited to: Eotvos Lorand University Budapest. Downloaded on October 27,2026 at 13:00:42 UTC from IEEE Xplore. Restrictions apply.

where $\mathbb{I}(a > b) = 1$ if $a > b$ and 0 otherwise. Thus, we can obtain the weighted cost for each task from the latency in (11), the energy consumption in (12), and the penalty in (13), i.e.,

$$C_{i,k} = \varphi_1 T_{i,k} + \varphi_2 E_{i,k} + \varphi_3 \Phi_{i,k} \tag{14}$$

where φ_1–φ_3 are the weighted parameters.

To minimize the weighted cost in (14) by jointly optimizing offloading decision $\boldsymbol{X} = \{\boldsymbol{x}_{i,k}, k \in \mathcal{K}_i(t), i \in \mathcal{I}, t \in \mathcal{T}\}$, V2I scheduling decision $\boldsymbol{Y} = \{\boldsymbol{y}_{i,k}, k \in \mathcal{K}_i(t), i \in \mathcal{I}, t \in \mathcal{T}\}$, V2V scheduling decision $\boldsymbol{Z} = \{\boldsymbol{z}_{i,k}, k \in \mathcal{K}_i(t), i \in \mathcal{I}, t \in \mathcal{T}\}$, computing capacity decision $\boldsymbol{F} = \{f_{i,k}^{\mathrm{HPC}}, f_{i,k,i'}^{\mathrm{LPC}}, k \in \mathcal{K}_i(t), i' \in \mathcal{I} \setminus \{i\}, i \in \mathcal{I}, t \in \mathcal{T}\}$ and transmit power decision $\boldsymbol{P} = \{p_{i,k}^{\mathrm{V2I}}, p_{i,k}^{\mathrm{V2V}}, k \in \mathcal{K}_i(t), i \in \mathcal{I}, t \in \mathcal{T}\}$, we can formulate the long-term optimization problem for the whole system as follows:

$$\begin{aligned}
&\min_{\boldsymbol{X},\boldsymbol{Y},\boldsymbol{Z},\boldsymbol{F},\boldsymbol{P}} \sum_{t\in\mathcal{T}} \sum_{i\in\mathcal{I}} \sum_{k\in\mathcal{K}_i(t)} C_{i,k} \\
&\text{s.t. } x_{i,k}^{\mathrm{drop}} \in \{0,1\},\ x_{i,k}^{\mathrm{loc}} \in \{0,1\} \\
&\quad x_{i,k}^{\mathrm{V2I}} \in \{0,1\},\ x_{i,k}^{\mathrm{V2V}} \in \{0,1\} \\
&\quad y_{i,k,m,j} \in \{0,1\}, z_{i,k,i'} \in \{0,1\} \\
&\quad f_{i,k}^{\mathrm{HPC}} \in \left[0, \eta_1 f_i^{\max}\right],\ f_{i,k,i'}^{\mathrm{LPC}} \in \left[0, \eta_2 f_i^{\max}\right] \\
&\quad p_{i,k}^{\mathrm{V2I}} \in \left[0, p_i^{\mathrm{V2Imax}}\right],\ p_{i,k}^{\mathrm{V2V}} \in \left[0, p_i^{\mathrm{V2Vmax}}\right] \\
&\quad (1)-(5).
\end{aligned} \tag{15}$$

It is easy to see that the formulated problem is very challenging to solve due to the following reasons. First, the formulated problem is nonlinear mixed integer and long-term optimization, which is unable to solved efficiently by some classical optimization methods. Second, for each task, the offloading decision is made when the task arrives, the scheduling decision and the transmit power decision are made when all previous tasks in the transmission queue have been transmitted, while the computing capacity decision is made when all previous tasks in the computation queue have been completed on vehicles. Such an asynchronous offloading, scheduling and resource allocation decisions for each task incurs extra challenges for vehicles to optimize these decisions jointly. Third, due to the fast and randomly changing wireless environment as well as the low-latency requirement of applications, vehicles need to make decisions in an online and distributed manner. To address these challenges, we will leverage the DRL approach to solve the formulated problem in an online and asynchronous manner.

V. Algorithm Design

In this section, we will provide the details of the multilayer DRL-based framework and its key components, and then propose a multilayer D3QN and PDQN-based algorithm (MLDPBA).

A. Multilayer DRL-Based Framework

As shown in Fig. 3, we design a multilayer DRL-based framework, where each vehicle incorporates three key modules to facilitate decision making at different layers: 1) offloading module; 2) scheduling module (including V2I and V2V scheduling modules); and 3) computing module (including HPC and LPC computing modules).

1) *Offloading Module (First Layer):* The offloading module employs one neural network architecture to determine the task offloading decision when a task arrives.
2) *Scheduling Module (Second Layer):* The V2I and V2V scheduling modules, respectively, employ another neural network architecture to select a target edge node and a target vehicle, along with channel selection and transmit power, for all offloaded tasks in the V2I-T and V2V-T queues.
3) *Computing Module (Third Layer):* The HPC and LPC computing modules derive their closed-form expressions from the corresponding convex optimization problem using classic optimization techniques.

As shown in Fig. 3, when a task arrives at one vehicle, the vehicle's offloading module outputs a binary offloading action. If the task is pushed into the vehicle's HPC queue, its HPC computing module allocates the computing capacity for the task, resulting in a reward (i.e., weighted cost) that is used to train the neural network of offloading module. If the task is pushed into the vehicle's V2I-T queue, its V2I scheduling module outputs the V2I scheduling and transmit power actions to schedule the task to a target edge node, resulting in a reward that is used to train the neural networks of both offloading and V2I scheduling modules. If the task is pushed into the vehicle's V2V-T queue, its V2V scheduling module outputs the V2V scheduling and transmit power actions to schedule the task to a target vehicle, and then the target vehicle's LPC computing module allocates the computing capacity for the task, resulting in a reward that is used to train the neural networks of both offloading and V2V scheduling modules.

B. Key Components

In the following, we define the state, action and reward for offloading, V2I scheduling, and V2V scheduling modules, respectively.

1) Offloading Module: The state, action, and reward of the offloading module are defined as follows.

1) *State:* When vehicle i generates a task $w_{i,k}, k \in \mathcal{K}_i(t)$ at time slot t (i.e., $\tau_{i,k}$), it can observe the following offloading state:

$$\begin{aligned}
\boldsymbol{s}_{i,k}^{\mathrm{off}} \triangleq \Big(&w_{i,k}, Q_i^{\mathrm{HPC}}(\tau_{i,k}), Q_i^{\mathrm{V2I}}(\tau_{i,k}), Q_i^{\mathrm{V2V}}(\tau_{i,k}) \\
&Q_{i'}^{\mathrm{LPC}}(\tau_{i,k}), Q_j^{\mathrm{ENC}}(\tau_{i,k}), H_{i,m,j}(\tau_{i,k}), H_{i,i'}(\tau_{i,k}) \\
&i' \in \mathcal{I} \setminus \{i\}, j \in \mathcal{J}, m \in \mathcal{M}_j\Big).
\end{aligned}$$

2) *Action:* When vehicle i observes the current offloading state, it needs to immediately make the offloading action for task $w_{i,k}$

$$\boldsymbol{a}_{i,k}^{\mathrm{off}} \triangleq \boldsymbol{x}_{i,k} = \left(x_{i,k}^{\mathrm{dro}}, x_{i,k}^{\mathrm{loc}}, x_{i,k}^{\mathrm{V2I}}, x_{i,k}^{\mathrm{V2V}}\right).$$

3) *Reward:* We know from Fig. 3 that the reward of task $w_{i,k}$ may not be achieved at time slot t (i.e., $\tau_{i,k}$) immediately, but will be achieved when it is fully completed in the future. Hence, we define the reward as follows:

$$R_{i,k} \triangleq -C_{i,k}.$$

Authorized licensed use limited to: Eotvos Lorand University Budapest. Downloaded on October 27,2026 at 13:00:42 UTC from IEEE Xplore. Restrictions apply.

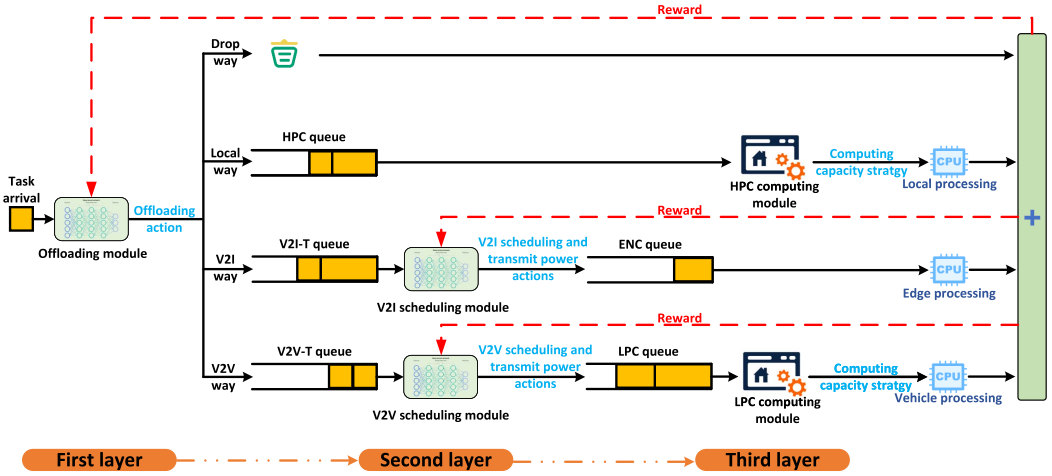


Fig. 3. Illustration of multilayer DRL-based framework.

2) V2I Scheduling Module: The state, action, and reward of the V2I scheduling module are defined as follows.

1) *State:* When a task $w_{i,k}$ in the V2I-T queue of vehicle i is ready for data transmission at time $t_1^{\mathrm{V2I}} = \tau_{i,k} + T_{i,k}^{\mathrm{V2Iw}}$, it can observe the following V2I scheduling state:

$$\boldsymbol{s}_{i,k}^{\mathrm{V2I}} \triangleq \left(w_{i,k}, Q_j^{\mathrm{ENC}}\left(t_1^{\mathrm{V2I}}\right), H_{i,m,j}\left(t_1^{\mathrm{V2I}}\right), j \in \mathcal{J}, m \in \mathcal{M}_j\right).$$

2) *Action:* When vehicle i observes the current V2I scheduling state, it needs to make the V2I scheduling and transmit power actions for task $w_{i,k}$

$$\boldsymbol{a}_{i,k}^{\mathrm{V2I}} \triangleq \left(\boldsymbol{y}_{i,k}, p_{i,k}^{\mathrm{V2I}}\right).$$

3) *Reward:* When task $w_{i,k}$ has been pushed into the ENC queue of a target edge node and has been fully completed, the reward will be achieved in the future. Hence, we define the reward as follows:

$$R_{i,k} \triangleq -C_{i,k}.$$

3) V2V Scheduling Module: The state, action, and reward of the V2V scheduling module are defined as follows.

1) *State:* When a task $w_{i,k}$ in the V2V-T queue of vehicle i is ready for data transmission at time $t_1^{\mathrm{V2V}} = \tau_{i,k} + T_{i,k}^{\mathrm{V2Vw}}$, it can observe the following V2V scheduling state:

$$\boldsymbol{s}_{i,k}^{\mathrm{V2V}} \triangleq \left(w_{i,k}, Q_{i'}^{\mathrm{HPC}}\left(t_1^{\mathrm{V2V}}\right), Q_{i'}^{\mathrm{LPC}}\left(t_1^{\mathrm{V2V}}\right), H_{i,k,i'}\left(t_1^{\mathrm{V2V}}\right) i' \in \mathcal{I} \setminus \{i\}\right).$$

2) *Action:* When vehicle i observes the current V2V scheduling state, it needs to make the V2V scheduling and transmit power actions for task $w_{i,k}$

$$\boldsymbol{a}_{i,k}^{\mathrm{V2V}} \triangleq \left(\boldsymbol{z}_{i,k}, p_{i,k}^{\mathrm{V2V}}\right).$$

3) *Reward:* When task $w_{i,k}$ has been pushed into the LPC queue of a target vehicle and has been fully completed, the reward will be achieved in the future. Hence, we define the reward as follows:

$$R_{i,k} \triangleq -C_{i,k}.$$

C. Multilayer D3QN and PDQN-Based Algorithm

To effectively address the challenges posed by asynchronous decision making and hybrid discrete-continuous decisions processes, we adopt two distinct neural network architectures. For the offloading module, we employ the D3QN framework [42] as one neural network architecture of offloading module For the scheduling module, we employ PDQN framework [43] as another neural network architecture of scheduling module. In the following, we first provide the action selection and weight updating mechanisms for offloading, V2I scheduling, and V2V scheduling modules. Next, we formulate the computing resource allocation problem as a convex optimization problem, and then derive the closed-form expression of the optimal solution using classic optimization techniques through the computing module. Finally, we illustrate the details of Algorithm 1.

1) Offloading Module: Given a state $\boldsymbol{s}^{\mathrm{off}}$ and an action $\boldsymbol{a}^{\mathrm{off}}$, the Q-value is defined as $Q(\boldsymbol{s}^{\mathrm{off}}, \boldsymbol{a}^{\mathrm{off}})$. We introduce one evaluation network for approximating the Q-value as $Q(\boldsymbol{s}^{\mathrm{off}}, \boldsymbol{a}^{\mathrm{off}}; \boldsymbol{\theta}^{\mathrm{off}}) \approx Q(\boldsymbol{s}^{\mathrm{off}}, \boldsymbol{a}^{\mathrm{off}})$ with its weights $\boldsymbol{\theta}^{\mathrm{off}}$. When vehicle i observes a state $\boldsymbol{s}_{i,k}^{\mathrm{off}}$ for task $w_{i,k}$, the vehicle's offloading module needs to choose an action $\boldsymbol{a}_{i,k}^{\mathrm{off}}$, i.e.,

$$\boldsymbol{a}_{i,k}^{\mathrm{off}} = \begin{cases} \text{Sample a random action } w.p.\ \epsilon_i^{\mathrm{off}}, \\ \arg\max_{\boldsymbol{a}} Q\left(\boldsymbol{s}_{i,k}^{\mathrm{off}}, \boldsymbol{a}; \boldsymbol{\theta}_i^{\mathrm{off}}\right) \quad w.p.\ 1 - \epsilon_i^{\mathrm{off}} \end{cases} \tag{16}$$

where $\epsilon_i^{\mathrm{off}}$ is the probability of random exploration and decays over time, and $\boldsymbol{\theta}_i^{\mathrm{off}}$ is the weights of the offloading module of vehicle i. Let a set $\mathcal{B}_i^{\mathrm{off}}$ of vehicle i denote the replay buffer to store the experiences. To be specific, the oth experience is represented as $(\boldsymbol{s}^{\mathrm{off}}(o), \boldsymbol{a}^{\mathrm{off}}(o), R(o), \boldsymbol{s}^{\mathrm{off}}(o+1))$, where $R(o)$ and $\boldsymbol{s}^{\mathrm{off}}(o+1)$ are the reward and the next state. A batch of experiences $\mathcal{D}_i^{\mathrm{off}}$ are randomly sampled from the replay buffer $\mathcal{B}_i^{\mathrm{off}}$ of vehicle i. Thus, the loss function for updating the weights $\boldsymbol{\theta}_i^{\mathrm{off}}$ can be defined as follows:

$$L\left(\boldsymbol{\theta}_i^{\mathrm{off}}\right) = \frac{1}{|\mathcal{D}_i^{\mathrm{off}}|} \sum_{o \in \mathcal{D}_i^{\mathrm{off}}} \left(Q\left(\boldsymbol{s}^{\mathrm{off}}(o), \boldsymbol{a}^{\mathrm{off}}(o); \boldsymbol{\theta}_i^{\mathrm{off}}\right) - y_o^{\mathrm{off}}\right)^2 \tag{17}$$

Authorized licensed use limited to: Eotvos Lorand University Budapest. Downloaded on October 27,2026 at 13:00:42 UTC from IEEE Xplore. Restrictions apply.

where $y_o^{\text{off}} = R(o) + \gamma Q(\boldsymbol{s}^{\text{off}}(o+1), \boldsymbol{a}_{o+1}; \hat{\boldsymbol{\theta}}_i^{\text{off}})$, $\boldsymbol{a}_{o+1} = \arg\max_{\boldsymbol{a}} Q(\boldsymbol{s}^{\text{off}}(o+1), \boldsymbol{a}; \boldsymbol{\theta}_i^{\text{off}})$, and $\hat{\boldsymbol{\theta}}_i^{\text{off}}$ is the weights of the target network. $|\mathcal{D}_i^{\text{off}}|$ and γ are the cardinality of the set $\mathcal{D}_i^{\text{off}}$ and the discount factor.

2) V2I/V2V Scheduling Module: For notational convenience, we denote a V2I scheduling set and a transmit power set as $\mathcal{N}^{\text{V2I}} = \{1, 2, \ldots, \sum_{j\in\mathcal{J}} M_j\}$ and $\mathcal{P}_i^{\text{V2I}} = \{p^{\text{V2I}} | 0 < p^{\text{V2I}} \le p_i^{\text{V2Imax}}\}$, and denote a V2V scheduling set and a transmit power set as $\mathcal{N}^{\text{V2V}} = \{1, 2, \ldots, I-1\}$ and $\mathcal{P}_i^{\text{V2V}} = \{p^{\text{V2V}} | 0 < p^{\text{V2V}} \le p_i^{\text{V2Vmax}}\}$ for vehicle $i \in \mathcal{I}$. Hence, the hybrid discrete-continuous action spaces of the V2I and V2V scheduling modules can be rewritten as the parameterized action spaces

$$\mathcal{A}_i^l = \left\{ \left(n^l, p^l\right) | n^l \in \mathcal{N}^l, p^l \in \mathcal{P}_i^l \right\}, \quad l \in \Lambda = \{\text{V2I}, \text{V2V}\}$$

where n^l is a particular discrete action, i.e., l scheduling decision with a certain channel, while p^l is a particular continuous action for l transmit power associated with the discrete actions n^l. Accordingly, the Q-value $Q(\boldsymbol{s}^l, \boldsymbol{a}^l)$ can be rewritten as $Q(\boldsymbol{s}^l, n^l, p_{n^l}^l)$. For a particular discrete action n^l, the continuous action $p_{n^l}^l$ can be evaluated in the following:

$$p_{n^l}^l\left(\boldsymbol{s}^l\right) = \arg\sup_{p\in\mathcal{P}} Q\left(\boldsymbol{s}^l, n^l, p\right)$$

which is a function of state $\boldsymbol{s}^l$. We introduce one neural network (called the critic) for approximating the Q-value as $Q(\boldsymbol{s}^l, n^l, p_{n^l}^l; \boldsymbol{\omega}^l) \approx Q(\boldsymbol{s}^l, n^l, p_{n^l}^l)$ with its weights $\boldsymbol{\omega}^l$. Moreover, we introduce another neural network (called the actor) for approximating the deterministic policy $p_{n^l}^l(\boldsymbol{s}^l; \boldsymbol{\varpi}^l) \approx p_{n^l}^l(\boldsymbol{s}^l)$ with its weights $\boldsymbol{\varpi}^l$, which outputs the continuous actions of all discrete actions.

For a given $l \in \Lambda$ scheduling module, when vehicle i observes a scheduling state $\boldsymbol{s}_{i,k}^l$ for task $w_{i,k}$, the l actor $p^l(\boldsymbol{s}_{i,k}^l; \boldsymbol{\varpi}_i^l)$ outputs the all continuous actions

$$\boldsymbol{p}_{i,k}^l = \{p_{i,k,1}^l, p_{i,k,2}^l, \ldots, p_{i,k,|\mathcal{N}^l|}^l\} \tag{18}$$

which is corresponding to the discrete actions $\{1, 2, \ldots, |\mathcal{N}^l|\}$, and $\boldsymbol{\varpi}_i^l$ is the weights of the l actor of vehicle i. Then it needs to choose a discrete action $n_{i,k}^l$ through the l critic, i.e.,

$$n_{i,k}^l = \begin{cases} \text{Sample a random action} & w.p.\ \epsilon_i^l \\ \arg\max\limits_{n^l\in\mathcal{N}^l} Q\left(\boldsymbol{s}_{i,k}^l, n^l, p_{i,k,n^l}^l; \boldsymbol{\omega}_i^l\right) & w.p.\ 1-\epsilon_i^l \end{cases} \tag{19}$$

where $\boldsymbol{\omega}_i^l$ is the weights of the l actor of vehicle i. Thus, we can obtain the action $\boldsymbol{a}_{i,k}^l$ from (18) and (19), i.e.,

$$\boldsymbol{a}_{i,k}^l = \left(n_{i,k}^l, p_{i,k,n_{i,k}^l}^l\right). \tag{20}$$

Similarly, based on the sampled experiences $\mathcal{D}_i^l$ from the replay buffer $\mathcal{B}_i^l$, the loss function for updating the weights $\boldsymbol{\omega}_i^l$ of the l critic can be defined as follows:

$$L\left(\boldsymbol{\omega}_i^l\right) = \frac{1}{|\mathcal{D}_i^l|} \sum_{o\in\mathcal{D}_i^l} \left(Q\left(\boldsymbol{s}^l(o), n^l(o), p^l(o); \boldsymbol{\omega}_i^l\right) - y_o^l\right)^2 \tag{21}$$

where $y_o^l = R(o) + \gamma \max_{n^l\in\mathcal{N}^l} Q_i(\boldsymbol{s}^l(o+1), n^l, p_i^l(\boldsymbol{s}(o+1); \hat{\boldsymbol{\varpi}}_i^l); \hat{\boldsymbol{\omega}}_i^l)$, and $\hat{\boldsymbol{\varpi}}_i^l$ and $\hat{\boldsymbol{\omega}}_i^l$ are the target network's weights of the l actor and l critic of vehicle i. Furthermore, the objective function for updating the weights $\boldsymbol{\omega}_i^l$ of the l actor can be defined as follows:

$$J\left(\boldsymbol{\varpi}_i^l\right) = -\frac{1}{|\mathcal{D}_i^l|} \sum_{o\in\mathcal{D}_i^l} Q\left(\boldsymbol{s}^l(o), n^l(o), p_i^l(\boldsymbol{s}^l(o); \boldsymbol{\varpi}_i^l); \boldsymbol{\omega}_i^l\right). \tag{22}$$

3) HPC/LPC Computing Module: As shown in Fig. 3, the computing module is responsible for allocating the computing capacity for local processing and vehicle processing in the local way and the V2V way, respectively.

In the local way, when a task $w_{i,k}$ of vehicle i is pulled out from the HPC queue of the vehicle at time t_1^{loc} (mentioned in Section III-E), it will observe an ordered task set $Q_i^{\text{HPC}}(t_1^{\text{loc}}) = \{k_1, k_2, \ldots, k_{\tilde{N}_{i,k}}\}$, where $\tilde{N}_{i,k}$ represents the number of tasks in the set. To determine the computing capacity $f_{i,k}^{\text{HPC}}$, we formulate the computing resource allocation problem as follows:

$$\begin{aligned} \min_f \ & \varphi_1 \sum_{n=1}^{\tilde{N}_{i,k}} \left(t_1^{\text{loc}} - \tau_{i,k_n} + \sum_{n'=1}^{n} \frac{c_{i,k_{n'}}}{f} + \frac{c_{i,k}}{f}\right) \\ & + \varphi_2 \xi_i f^2 \left(\sum_{n=1}^{\tilde{N}_{i,k}} c_{i,k_n} + c_{i,k}\right) \\ \text{s. t.} \quad & f \in \left[0, \eta_1 f_i^{\max}\right]. \end{aligned} \tag{23}$$

Here, the first term represents the total latency, which includes both the waiting time and computation time for all tasks in the queue $Q_i^{\text{HPC}}(t_1^{\text{loc}})$, while the second term accounts for the total energy consumption of all tasks in the same queue.

By applying the first-order necessary condition to the problem (23), we derive the following theorem, which provides a closed-form expression $f_{i,k}^{\text{HPC}}$ for task $w_{i,k}$.

Theorem 1: The optimal solution $f_{i,k}^{\text{HPC}}$ for task $w_{i,k}$ by solving the problem (23) is given by (24), shown at the bottom of the next page.

In the V2V way, when a task $w_{i,k}$ of vehicle i is pulled out from the LPC queue of vehicle i' at time t_3^{V2V}, it will observe the ordered task set $Q_{i'}^{\text{LPC}}(t_3^{\text{V2V}}) = \{(i_1, k_1), (i_2, k_2), \ldots, (i_{\tilde{N}_{i,k}}, k_{\tilde{N}_{i,k}})\}$, where $\tilde{N}_{i,k}$ is the number of tasks in the set. Similarly, we utilize the same analysis technique of the local way to obtain the computing capacity $f_{i,k,i'}^{\text{LPC}}$ given by (25), shown at the bottom of the next page.

4) Algorithm Description: As shown in Algorithm 1, we first initialize the weights of the evaluation network, V2I/V2V actor, V2I/V2V critic, and their target networks, as well as the corresponding replay buffers in steps 1–4. In step 5, temporary variables Ξ_i^{H}, Ξ_i^{L}, Ξ_i^{I}, and Ξ_i^{V} are initialized for HPC/LPC computing module and V2I/V2V scheduling module to record the task's information. In step 6, E represents the number of episode, each containing a set $\{1, 2, \ldots, T\}$ of T time slot. For a given vehicle i in step 10, vehicle i generates the task set $\mathcal{K}_i(t)$ with Poisson distribution at time slot t. Vehicle i observes the offloading state $\boldsymbol{s}_{i,k}^{\text{off}}$ for task $w_{i,k} \in \mathcal{K}_i(t)$ and then makes the offloading action $\boldsymbol{a}_{i,k}^{\text{off}}$ according to (16) in steps 12 and 13. This implies that the task is either pushed into the corresponding queue if processed, or pushed into the set $\Theta(t)$ if dropped, where $\Theta(t)$ is a set to record the finished or dropped tasks. Once the tasks in $\mathcal{K}_i(t)$ are sequentially pulled out, vehicle i proceeds to activate the modules (namely, HPC/LPC computing module and V2I/V2V

Authorized licensed use limited to: Eotvos Lorand University Budapest. Downloaded on October 27,2026 at 13:00:42 UTC from IEEE Xplore. Restrictions apply.

Algorithm 1: Multilayer DQN and PDQN-Based Algorithm

Initialize the evaluation network $Q(\boldsymbol{s}^{\text{off}}, \boldsymbol{a}^{\text{off}}; \boldsymbol{\theta}_i^{\text{off}})$ with random weights $\boldsymbol{\theta}_i^{\text{off}}$, $i \in \mathcal{I}$;
Initialize the l actor $p^l(\boldsymbol{s}^l; \boldsymbol{\varpi}_i^l)$ with random weights $\boldsymbol{\varpi}_i^l$ and the l critic $Q(\boldsymbol{s}^l, n^l, p_{n^l}^l; \boldsymbol{\omega}_i^l)$ with random weights $\boldsymbol{\omega}_i^l$, $l \in \Lambda$, $i \in \mathcal{I}$;
Initialize the corresponding target networks;
Initialize replay buffers $\mathcal{B}_i^{\text{off}}$, $\mathcal{B}_i^l$, $l \in \Lambda$, $i \in \mathcal{I}$;
Initialize temporary variables: $\Xi_i^{\text{H}} := \text{Nan}$, $\Xi_i^{\text{I}} := \text{Nan}$, $\Xi_i^{\text{V}} := \text{Nan}$, $\Xi_i^{\text{L}} := \text{Nan}$, and set $Count := 1$;
for *episode* $= 1, 2, \ldots, E$ **do**
  Reset the MEC-assisted VEN;
  for *time slot* $t = 1, \ldots, T$ **do**
    # Collect the experiments;
    for *vehicle* $i = 1, 2, \ldots, I$ **do**
      for *task* $w_{i,k}, k \in \mathcal{K}_i(t)$ **do**
        Observe the state $\boldsymbol{s}_{i,k}^{\text{off}}$ for task $w_{i,k}$;
        Make the action $\boldsymbol{a}_{i,k}^{\text{off}}$ from (16);
        Add the task to set $\Theta(t)$ or push the task into the corresponding queue;
      end
      HPC/LPC Computing Module (Algorithm 2);
      V2I/V2V Scheduling Module (Algorithm 3);
    end
    Reward Collection (Algorithm 4);
    # Training the neural networks;
    for *vehicle* $i = 1, 2, \ldots, I$ **do**
      if *mod(Count, Update_Thr1)=0* **then**
        Sample a batch of experiences;
        Update the weights $\boldsymbol{\theta}_i^{\text{off}}$ by (17);
        Update the weights $\boldsymbol{\omega}_i^l$, $l \in \Lambda$ by (21);
        Update the weights $\boldsymbol{\varpi}_i^l$, $l \in \Lambda$ by (22);
      end
      if *mod(Count, Update_Thr2)=0* **then**
        Update the weights of target networks;
      end
    end
    $Count := Count + 1$;
  end
end

Algorithm 2: HPC Computing Module

1 Reset the remaining time: $t_{re}^{\text{H}} := \Delta$;
2 **while** *True* **do**
3 **if** $\Xi_i^{\text{H}} = \text{Nan}$ *and* $Q_i^{\text{HPC}} \neq \emptyset$ **then**
4 Pull out a task $w_{i,k'}$ from HPC queue Q_i^{HPC};
5 Evaluate the computing capacity $f_{i,k'}^{\text{HPC}}$ from (24);
6 Set $\Xi_i^{\text{H}}.ta = w_{i,k'}$, $\Xi_i^{\text{H}}.cpu = f_{i,k'}^{\text{HPC}}$, $\Xi_i^{\text{H}}.re_c = c_{i,k'}$;
7 **end**
8 **if** $\Xi_i^{\text{H}} \neq \text{Nan}$ *and* $\Xi_i^{\text{H}}.re_c > 0$ **then**
9 **if** $\Xi_i^{\text{H}}.re_c \geq \Xi_i^{H}.cpu * t_{re}^{H}$ **then**
10 $\Xi_i^{\text{H}}.re_c -= \Xi_i^{\text{H}}.cpu * t_{re}^{\text{H}}$ and set $t_{re}^{\text{H}} = 0$;
11 **else**
12 $t_{re}^{\text{H}} -= \Xi_i^{\text{H}}.re_c / \Xi_i^{\text{H}}.cpu$ and reset $\Xi_i^{\text{H}} = \text{Nan}$;
13 Add the task $\Xi_i^{\text{H}}.ta$ to set $\Theta(t)$;
14 **end**
15 **end**
16 **if** $t_{re}^{\text{H}} \leq 0$ *or* $Q_i^{\text{HPC}} = \emptyset$ **then**
17 break;
18 **end**
19 **end**

scheduling module) to process the tasks in their queues over a duration (i.e., Δ seconds).

For HPC Computing Module in Algorithm 2, vehicle i resets the remaining time $t_{re}^{\text{H}} := \Delta$ in step 1. If the temporary variable Ξ_i^{H} equals Nan and the HPC queue Q_i^{HPC} is not empty, the vehicle pulls out a task $w_{i,k'}$ from the HPC queue, evaluates the computing capacity $f_{i,k'}^{\text{HPC}}$ for task $w_{i,k'}$ according to (24), and records the current task's information as follows: $\Xi_i^{\text{H}}.ta = w_{i,k'}$, $\Xi_i^{\text{H}}.cpu = f_{i,k'}^{\text{HPC}}$, and $\Xi_i^{\text{H}}.re_c = c_{i,k'}$. Here, Ξ_i^{H} comprises three attributions: task *ta*, computing capacity *cpu*, and remaining cycle *re_c*. If Ξ_i^{H} does not equal Nan and the remaining cycle $\Xi_i^{\text{H}}.re_c$ is greater than zero, the following operations occur: If the remaining cycle $\Xi_i^{\text{H}}.re_c$ can be fully processed within the remaining time t_{re}^{H}, the updated remaining time becomes $t_{re}^{\text{H}} -= \Xi_i^{\text{H}}.re_c / \Xi_i^{\text{H}}.cpu$, i.e., $t_{re}^{\text{H}} = t_{re}^{\text{H}} - \Xi_i^{\text{H}}.re_c / \Xi_i^{\text{H}}.cpu$, the temporary variable Ξ_i^{H} is reset to Nan, and the completed task $\Xi_i^{\text{H}}.ta$ is added to the set $\Theta(t)$ in steps 12 and 13. Otherwise, the remaining cycle is updated as $\Xi_i^{\text{H}}.re_c -= \Xi_i^{\text{H}}.cpu * t_{re}^{\text{H}}$, and the remaining time t_{re}^{H} is reset to zero in step 10. Finally, if the remaining time t_{re}^{H} is exhausted or the HPC queue is empty, the HPC computing module is exited; otherwise, continue to step 2. The LPC Computing Module follows the same operational procedures as the HPC Computing Module.

For V2I Scheduling Module in Algorithm 3, vehicle i resets the remaining time $t_{re}^{\text{I}} := \Delta$ in step 1. If the temporary variable

$$f_{i,k}^{\text{HPC}} = \min\left\{\eta_1 f_i^{\max}, \sqrt{\varphi_1 \sum_{n=1}^{\tilde{N}_{i,k}} \left(\sum_{n'=1}^{n} c_{i,k_{n'}} + c_{i,k}\right) / 2\varphi_2 \xi_i \left(\sum_{n=1}^{\tilde{N}_{i,k}} c_{i,k_n} + c_{i,k}\right)}\right\} \tag{24}$$

$$f_{i,k,i'}^{\text{LPC}} = \min\left\{\eta_2 f_{i'}^{\max}, \sqrt{\varphi_1 \sum_{n=1}^{\tilde{N}_{i,k}} \left(\sum_{n'=1}^{n} c_{i_{n'},k_{n'}} + c_{i,k}\right) / 2\varphi_2 \xi_{i'} \left(\sum_{n=1}^{\tilde{N}_{i,k}} c_{i_n,k_n} + c_{i,k}\right)}\right\} \tag{25}$$

Authorized licensed use limited to: Eotvos Lorand University Budapest. Downloaded on October 27,2026 at 13:00:42 UTC from IEEE Xplore. Restrictions apply.

Algorithm 3: V2I Scheduling Module

1 Reset the remaining time: $t_{re}^{\mathrm{I}} := \Delta$;
2 **while** *True* **do**
3 **if** $\Xi_i^{\mathrm{I}} =$ Nan *and* $Q_i^{\mathrm{V2I}} \neq \emptyset$ **then**
4 Pull out a task $w_{i,k'}$ from V2I-T queue Q_i^{V2I};
5 Observe the V2I scheduling state $\boldsymbol{s}_{i,k'}^{\mathrm{V2I}}$;
6 Make the V2I scheduling action $\boldsymbol{a}_{i,k'}^{\mathrm{V2I}}$ from (20);
7 Set $\Xi_i^{\mathrm{I}}.ta = w_{i,k'}$ and $\Xi_i^{\mathrm{I}}.re_b = d_{i,k'}$;
8 **end**
9 **if** $\Xi_i^{\mathrm{I}} \neq$ Nan *and* $\Xi_i^{\mathrm{I}}.re_b > 0$ **then**
10 Evaluate the rate r^{I} for task $\Xi_i^{\mathrm{I}}.ta$ from (6);
11 **if** $\Xi_i^{\mathrm{I}}.re_b \geq r^{\mathrm{I}} * t_{re}^{\mathrm{I}}$ **then**
12 $\Xi_i^{\mathrm{I}}.re_b -= r^{\mathrm{I}} * t_{re}^{\mathrm{I}}$ and set $t_{re}^{\mathrm{I}} = 0$;
13 **else**
14 $t_{re}^{\mathrm{I}} -= \Xi_i^{\mathrm{I}}.re_b / r^{\mathrm{I}}$ and reset $\Xi_i^{\mathrm{I}} =$ Nan;
15 Send the task $\Xi_i^{\mathrm{I}}.ta$ to the target edge node;
16 Process the task $\Xi_i^{\mathrm{I}}.ta$ at the target edge node;
17 Add the task $\Xi_i^{\mathrm{I}}.ta$ to set $\Theta(t)$;
18 **end**
19 **end**
20 **if** $t_{re}^{\mathrm{I}} \leq 0$ *or* $Q_i^{\mathrm{V2I}} = \emptyset$ **then**
21 break;
22 **end**
23 **end**

Algorithm 4: Reward Collection

1 **for** $w_{\tilde{i},\tilde{k}} \in \Theta(t)$ **do**
2 Obtain the reward $R_{\tilde{i},\tilde{k}}$ for task $w_{\tilde{i},\tilde{k}}$;
3 **if** $x_{\tilde{i},\tilde{k}}^{\mathrm{loc}} = 1$ *or* $x_{\tilde{i},\tilde{k}}^{\mathrm{dro}} = 1$ **then**
4 Store the experience $(\boldsymbol{s}_{\tilde{i},\tilde{k}}^{\mathrm{off}}, \boldsymbol{a}_{\tilde{i},\tilde{k}}^{\mathrm{off}}, R_{\tilde{i},\tilde{k}}, \boldsymbol{s}_{-\tilde{i},\tilde{k}}^{\mathrm{off}})$ into replay buffer $\mathcal{B}_{\tilde{i}}^{\mathrm{off}}$;
5 **else if** $x_{\tilde{i},\tilde{k}}^{\mathrm{V2I}} = 1$ **then**
6 Store the experience $(\boldsymbol{s}_{\tilde{i},\tilde{k}}^{\mathrm{off}}, \boldsymbol{a}_{\tilde{i},\tilde{k}}^{\mathrm{off}}, R_{\tilde{i},\tilde{k}}, \boldsymbol{s}_{-\tilde{i},\tilde{k}}^{\mathrm{off}})$ into replay buffer $\mathcal{B}_{\tilde{i}}^{\mathrm{off}}$;
7 Store the experience $(\boldsymbol{s}_{\tilde{i},\tilde{k}}^{\mathrm{V2I}}, \boldsymbol{a}_{\tilde{i},\tilde{k}}^{\mathrm{V2I}}, R_{\tilde{i},\tilde{k}}, \boldsymbol{s}_{-\tilde{i},\tilde{k}}^{\mathrm{V2I}})$ into replay buffer $\mathcal{B}_{\tilde{i}}^{\mathrm{V2I}}$;
8 **else if** $x_{\tilde{i},\tilde{k}}^{\mathrm{V2V}} = 1$ **then**
9 Store the experience $(\boldsymbol{s}_{\tilde{i},\tilde{k}}^{\mathrm{off}}, \boldsymbol{a}_{\tilde{i},\tilde{k}}^{\mathrm{off}}, R_{\tilde{i},\tilde{k}}, \boldsymbol{s}_{-\tilde{i},\tilde{k}}^{\mathrm{off}})$ into replay buffer $\mathcal{B}_{\tilde{i}}^{\mathrm{off}}$;
10 Store the experience $(\boldsymbol{s}_{\tilde{i},\tilde{k}}^{\mathrm{V2V}}, \boldsymbol{a}_{\tilde{i},\tilde{k}}^{\mathrm{V2V}}, R_{\tilde{i},\tilde{k}}, \boldsymbol{s}_{-\tilde{i},\tilde{k}}^{\mathrm{V2V}})$ into replay buffer $\mathcal{B}_{\tilde{i}}^{\mathrm{V2V}}$;
11 **end**
12 **end**

Ξ_i^{I} equals Nan and the V2I-T queue Q_i^{V2I} is not empty, the vehicle pulls out a task $w_{i,k'}$ from the V2I-T queue, observes the V2I scheduling state $\boldsymbol{s}_{i,k'}^{\mathrm{V2I}}$, makes the V2I scheduling action $\boldsymbol{a}_{i,k'}^{\mathrm{V2I}}$ according to (6), and records the current task' information as follows: $\Xi_i^{\mathrm{I}}.ta = w_{i,k'}$ and $\Xi_i^{\mathrm{I}}.re_b = d_{i,k'}$. Here, Ξ_i^{I} comprises two attributions: task *ta* and remaining bit *re_b*. If Ξ_i^{I} does not equal Nan and the remaining bit $\Xi_i^{\mathrm{I}}.re_b$ is greater than zero, the vehicle evaluates the V2I transmission rate r^{I} for task $\Xi_i^{\mathrm{I}}.ta$ according to (6) in step 10. If the remaining bit $\Xi_i^{\mathrm{I}}.re_b$ can be fully processed within the remaining time t_{re}^{I}, then the remaining time is updated as $t_{re}^{\mathrm{I}} -= \Xi_i^{\mathrm{I}}.re_b/r^{\mathrm{I}}$ and Ξ_i^{H} is reset to Nan. Next, task $\Xi_i^{\mathrm{I}}.ta$ is sent to the target edge node, waits to be processed in the queue of the target edge node, and is added to the set $\Theta(t)$ in steps 14–17. Otherwise, the remaining bit is updated as $\Xi_i^{\mathrm{I}}.re_b -= r^{\mathrm{I}} * t_{re}^{\mathrm{I}}$, and the remaining time t_{re}^{I} is reset to zero in step 12. Finally, if the remaining time t_{re}^{I} is exhausted or the V2I-T queue is empty, the V2I scheduling module is exited; otherwise, continue in step 3. V2V Scheduling Module follows the same operational procedures as the V2I Scheduling Module.

For Reward Collection in Algorithm 4, for a given completed task $w_{\tilde{i},\tilde{k}} \in \Theta(t)$, vehicle $\tilde{i}$ will obtain the reward $R_{\tilde{i},\tilde{k}}$ for the task. If the task $w_{\tilde{i},\tilde{k}}$ is completed in the local way or the drop way, the experience $(\boldsymbol{s}_{\tilde{i},\tilde{k}}^{\mathrm{off}}, \boldsymbol{a}_{\tilde{i},\tilde{k}}^{\mathrm{off}}, R_{\tilde{i},\tilde{k}}, \boldsymbol{s}_{-\tilde{i},\tilde{k}}^{\mathrm{off}})$ is stored into replay buffer $\mathcal{B}_{\tilde{i}}^{\mathrm{off}}$. This is used to train the neural network of offloading module. If the task $w_{\tilde{i},\tilde{k}}$ is completed in the V2I way, the experience $(\boldsymbol{s}_{\tilde{i},\tilde{k}}^{\mathrm{off}}, \boldsymbol{a}_{\tilde{i},\tilde{k}}^{\mathrm{off}}, R_{\tilde{i},\tilde{k}}, \boldsymbol{s}_{-\tilde{i},\tilde{k}}^{\mathrm{off}})$ is stored into replay buffer $\mathcal{B}_{\tilde{i}}^{\mathrm{off}}$, while the experience $(\boldsymbol{s}_{\tilde{i},\tilde{k}}^{\mathrm{V2I}}, \boldsymbol{a}_{\tilde{i},\tilde{k}}^{\mathrm{V2I}}, R_{\tilde{i},\tilde{k}}, \boldsymbol{s}_{-\tilde{i},\tilde{k}}^{\mathrm{V2I}})$ is stored into replay buffer $\mathcal{B}_{\tilde{i}}^{\mathrm{V2I}}$. This is used to train the neural network of offloading module and V2I scheduling module. If the task $w_{\tilde{i},\tilde{k}}$ is completed in the V2V way, the experience $(\boldsymbol{s}_{\tilde{i},\tilde{k}}^{\mathrm{off}}, \boldsymbol{a}_{\tilde{i},\tilde{k}}^{\mathrm{off}}, R_{\tilde{i},\tilde{k}}, \boldsymbol{s}_{-\tilde{i},\tilde{k}}^{\mathrm{off}})$ is stored into replay buffer $\mathcal{B}_{\tilde{i}}^{\mathrm{off}}$, while the experience $(\boldsymbol{s}_{\tilde{i},\tilde{k}}^{\mathrm{V2V}}, \boldsymbol{a}_{\tilde{i},\tilde{k}}^{\mathrm{V2V}}, R_{\tilde{i},\tilde{k}}, \boldsymbol{s}_{-\tilde{i},\tilde{k}}^{\mathrm{V2V}})$ is stored into replay buffer $\mathcal{B}_{\tilde{i}}^{\mathrm{V2V}}$. This is used to train the neural network of offloading module and V2V scheduling module.

As shown in Algorithm 1, for every *Update_Thr*1 updates, the weights $\boldsymbol{\theta}_i^{\mathrm{off}}$, $\boldsymbol{\omega}_i^l$, and $\boldsymbol{\varpi}_i^l$, $l \in \Lambda$ of offloading module' evaluation network, and V2I and V2V scheduling modules' critic and actor are updated by (17), (21), and (22) based on the sampled experiences in steps 22–27. For every *Update_Thr*2 updates, the weights $\hat{\boldsymbol{\theta}}_i^{\mathrm{off}}$, $\hat{\boldsymbol{\omega}}_i^l$ and $\hat{\boldsymbol{\varpi}}_i^l$, $l \in \Lambda$ of the corresponding target networks are updated by the soft update strategy in steps 28–30, i.e., $\hat{\boldsymbol{\theta}}_i^{\mathrm{off}} = \eta^{\mathrm{off}} \hat{\boldsymbol{\theta}}_i \mathrm{off} + (1 - \eta^{\mathrm{off}}) \boldsymbol{\theta}_i^{\mathrm{off}}$, $\hat{\boldsymbol{\omega}}_i^l = \eta^l \hat{\boldsymbol{\omega}}_i^l + (1 - \eta^l) \boldsymbol{\omega}_i^l$, and $\hat{\boldsymbol{\varpi}}_i^l = \eta^l \hat{\boldsymbol{\varpi}}_i^l + (1 - \eta^l) \boldsymbol{\varpi}_i^l$.

It is important to note that in this multilayer algorithm, we intuitively treat the offloading, scheduling, and computing modules as integral components of a unified network. This approach emphasizes the collaboration and interdependencies among the modules, thereby enhancing the overall stability of the system. Furthermore, existing literature [38], [39], [40], [41] has consistently demonstrated the robustness of multilayer algorithms across various fields, providing both theoretical foundations and practical validation for the stability of collaborative decision making within these interconnected modules.

VI. Performance Evaluation

A. General Setups

In this section, we consider a 1000 m two-lane, one-way road equipped with multiple vehicles, multiple RSUs, and one MBS. The vehicles maintain a constant speed throughout.

Authorized licensed use limited to: Eotvos Lorand University Budapest. Downloaded on October 27,2026 at 13:00:42 UTC from IEEE Xplore. Restrictions apply.

TABLE III
PARAMETER SETTINGS

Parameter	Value
I, J, v_i, T, Δ, λ_i	8, 3, 10m/s, 100s, 1s, 0.7
$c_{i,k}$, $d_{i,k}$, $\delta_{i,k}$	[1, 7]Gcycles, [1, 7]Mbits, [1,2]s
η_1, η_2, $f_i^{\max}$	0.6, 0.4, 10MHz
M_j, M_{J+1}, p_i^{V2Imax}, p_i^{V2Vmax}	6, 8, 1w, 1w
f_j^{EN}, f_{J+1}^{EN},	30GHz, 100GHz
$W_{i'}$, W_j, W_{J+1}, σ^2,	5, 3, 3, -100dBm
ξ_i, μ, E	10^{-26}, 20, 2000
φ_1, φ_2, φ_3,	0.5, 0.5, 1
Learning rate, γ	0.001/0.0001/0.0001, 0.9
Hidden layers, Optimizer	[128, 128, 64], Adam

TABLE IV
BENCHMARK METHODS

Method	Description
Local Based Algorithm (LBA) [44]	Vehicles process their tasks locally
Random Based Algorithm (RBA) [44]	Vehicles randomly decide to drop, process locally, or offload tasks. Offloaded tasks are sent to a random target with a random transmit power.
Greedy Based Algorithm (GBA) [45]	Vehicles minimize weighted costs by offloading and scheduling tasks with maximum transmit powers at each time slot.
Single-Layer D3QN Based Algorithm (SLDBA) [46]	Vehicles adopt the D3QN approach to make offloading and scheduling decisions with discretized transmit powers.
Multi-Layer D3QN Based Algorithm (MLDBA) [38]	Vehicles adopt two neural networks within the D3QN framework for offloading and scheduling decisions.

RSUs are evenly spaced along the road, and the MBS is strategically positioned in the center. In these simulations, we consider four performance metrics: 1) average cost; 2) average latency (unit: s); 3) average energy (unit: J); and 4) average satisfaction rate. The average cost is evaluated based on the weighted cost of all tasks, while the average latency and average energy are the average values of completed task latency and energy consumption, respectively. The average satisfaction rate is defined as the average ratio of finished tasks to the total number of task arrivals over episodes. Unless stated otherwise, the basic parameter settings are listed in Table III. In addition, we compare our proposed MLDPBA algorithm with several benchmark methods, listed in Table IV.

B. Performance of Convergence

Fig. 4 illustrates the convergence of the proposed algorithm across episodes under different learning rates. The average reward is calculated by the negative weighted cost of all tasks generated by vehicles. Let O_lr denote the learning rate of the offloading module, while let V_a_lr and V_c_lr denote the learning rates of V2I/V2V actor and critic, respectively. The proposed algorithm can converge and achieve a higher reward when the learning rates are set to: O_lr $=$ 0.001 V_a_lr $=$ 0.0001 and V_c_lr $=$ 0.0001. In contrast, the other two learning rates can converge with lower rewards. This is because a larger learning rate leads to a higher level of fluctuation, causing the value of the loss function to oscillate around the minimum value.

Fig. 5(a) illustrates the convergence of MLDPBA algorithm and other benchmark methods across episodes in terms of

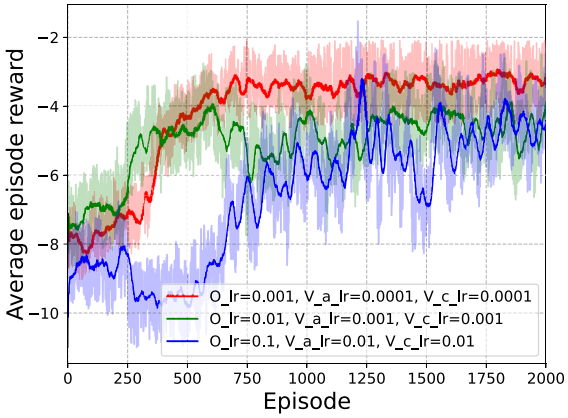


Fig. 4. Convergence of MLDPBA algorithm under different learning rates.

average cost. MLDPBA algorithm converges after approximately 1250 episodes, while MLDBA and SLDBA algorithms converge after approximately 1500 and 1750 episodes, respectively. To gain detailed insight into the performance of MLDPBA algorithm and other benchmark methods, Fig. 5(b) shows that the performance evaluation of average cost, average latency, average energy, and average satisfaction rate under different algorithms. We can infer from the results that MLDPBA algorithm outperforms all the other benchmark methods in terms of average cost, average latency, average energy and average satisfaction rate. This is because MLDPBA algorithm can smartly make intelligent decisions based on the new state and available information. To be specific, MLDPBA algorithm can reduce the average cost by 25.86%–72.51%, average latency by 9.61%–83.60%, and average energy consumption by 37.61%–83.67%. Moreover, it also can improve the average satisfaction rate by 3.48%–90.53%, when compared to other benchmark methods.

Fig. 6 illustrates the tradeoff of average latency and average energy under different weighted parameters $(\varphi_1, \varphi_2) = (0.9, 0.1), (0.7, 0.3), (0.5, 0.5), (0.3, 0.7), (0.1, 0.9)$, implying that this tradeoff yields the varying levels of average latency and average energy. Note that as the weighted parameter φ_1 decreases, average latency increases while average energy decreases, and vice versa. By adjusting the values of (φ_1, φ_2), vehicles can easily facilitate the proposed algorithm to meet their specific requirements, specifically for time-sensitivity and energy-efficiency.

C. Performance of Different Scenario Schemes

Fig. 7 illustrates the performance comparison of different scenario schemes based on the proposed algorithm. These scenario schemes include our scenario (OS), edge/cloud offloading mode-based scenario (ECOMS), and end-collaborative offloading mode-based scenario (EOMS), where tasks can be processed either on both vehicles and edge nodes, or only on edge nodes, or only on vehicles. The analysis of the results reveals that OS scheme reduces the average cost more effectively than the other two schems. To be specific, it lowers the average cost by 12.64% compared to ECOMS scheme and by 40.47% compared to EOMS scheme.

Authorized licensed use limited to: Eotvos Lorand University Budapest. Downloaded on October 27,2026 at 13:00:42 UTC from IEEE Xplore. Restrictions apply.

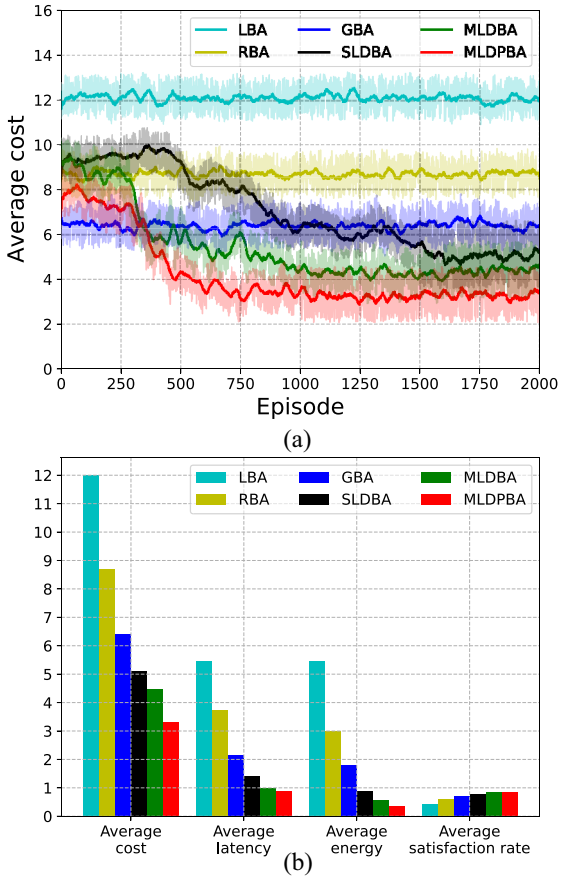


Fig. 5. Performance evaluation across episodes. (a) Average cost across episodes. (b) Performance evaluation under different algorithms.

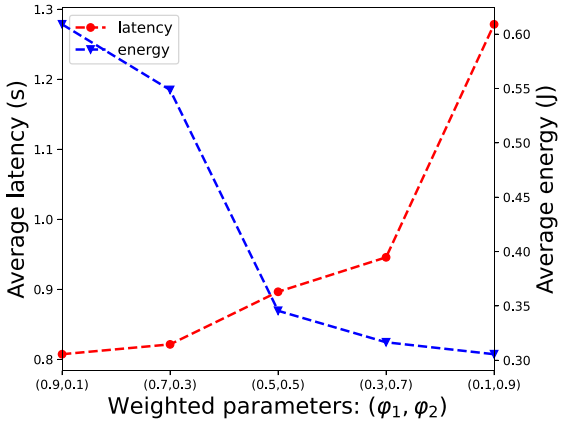


Fig. 6. Tradeoff of average latency and average energy.

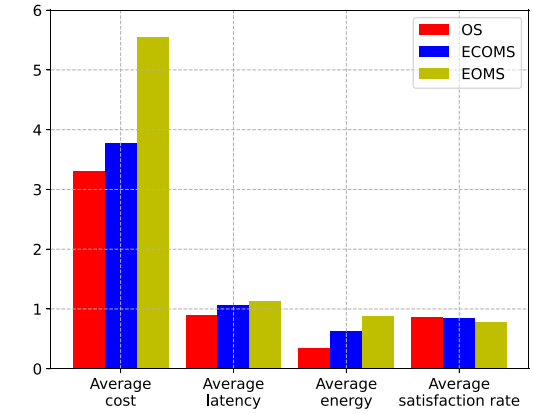


Fig. 7. Performance comparison of different scenario schemes.

Moreover, OS scheme leads to the reductions in both average latency and average energy. In this situation, the reductions are 15.09% and 45.18% when compared to ECOMS scheme, and 20.49% and 60.99% when compared to EOMS scheme. Furthermore, OS scheme has a higher average satisfaction rate than the other two schemes. These increment, respectively, are 1.80% and 11.46% when compared to ECOMS and EOMS schemes. These improvements are primarily attributed to the enhanced computing and communication resources in the OS scheme, which optimally leverages cloud–edge–end resources. By efficiently distributing workloads across cloud, edge and end-vehicles, OS scheme significantly boosts system responsiveness and energy efficiency.

D. Performance of Different System Parameters

1) Expectation λ: An increasing value of expectation λ indicates that a higher frequency of task generation by vehicles in MEC-enabled VENs. Fig. 8(a) illustrates the average cost of MLDPBA algorithm and other benchmark methods under different expectations λ. It is evident that MLDPBA algorithm consistently maintains a lower average cost compared to other benchmark methods as the expectation λ increases. Specifically, MLDPBA algorithm reduces the average cost by up to 82.22%, 77.34%, 74.23%, 69.15%, and 58.55% as the expectation λ ranges from 0.2 to 1.0. Furthermore, Fig. 8(b) illustrates the average satisfaction rate of MLDPBA algorithm and other benchmark methods under different expectations λ. The average satisfaction rate of all algorithms decreases significantly with the increasing expectations λ. Notably, MLDPBA algorithm outperforms other benchmark methods with a higher average satisfaction rate of approximately 86.29% across the expectation range of 0.2 to 1.0. In contrast, the average satisfaction rates of MLDBA, SLDBA, GBA, RBA, and LBA methods are around 83.43%, 81.40%, 78.82%, 58.69%, and 42.13%, respectively.

2) Maximum Data Size $d_i^{\max}$: An increase in the maximum data size d_i^{max} results in a higher system workload, thereby increasing the time and energy required for each tasks' completion. Fig. 9(a) illustrates the average cost of MLDPBA algorithm and other benchmark methods under different maximum data sizes $d_i^{\max}$. LBA algorithm incurs a higher average cost than other methods due to its limited computing capacity, which struggles to handle larger data sizes efficiently. In comparison, RBA algorithm achieves a lower average cost by leveraging both vehicles and edge nodes, effectively reducing latency and energy consumption through distributed processing. Furthermore, MLDPBA algorithm achieves a lowest average cost in comparison to other benchmark methods. Specifically, MLDPBA algorithm decreases the average cost by up to 86.52%, 75.22%, 72.74%, 64.95%, and 58.63% as the maximum data size ranges from 5 to 9. Fig. 9(b) illustrates the average satisfaction rate of

Authorized licensed use limited to: Eotvos Lorand University Budapest. Downloaded on October 27,2026 at 13:00:42 UTC from IEEE Xplore. Restrictions apply.

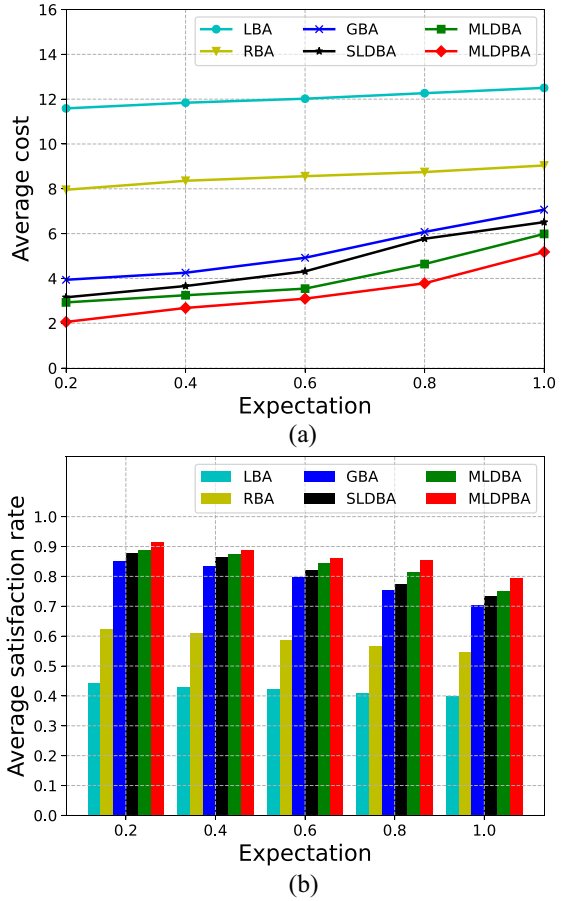


Fig. 8. Performance evaluation under different expectations λ. (a) Average cost. (b) Average satisfaction rate.

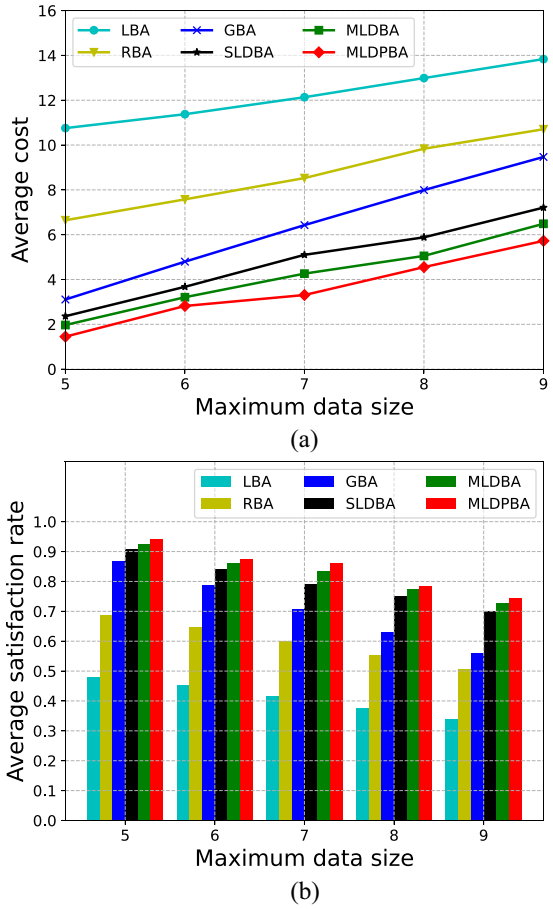


Fig. 9. Performance evaluation under different maximum data sizes $d_i^{\max}$. (a) Average cost. (b) Average satisfaction rate.

MLDPBA algorithm and other benchmark methods under different maximum data sizes $d_i^{\max}$. The average satisfaction rate of MLDPBA algorithm is significantly higher than that of other benchmark methods for the same maximum data size $d_i^{\max}$. This is due to MLDPBA algorithm's ability to explore the continuous action space and make accurate decisions to obtain an optimal policy, thereby maximizing satisfaction even under higher workloads. Remarkably, the average satisfaction rate of MLDPBA, MLDBA, SLDBA, GBA, RBA, and LBA algorithms are approximately 84.15%, 82.50%, 79.93%, 71.05%, 59.96%, and 41.30% across the maximum data size range of 5 to 9.

3) Maximum Deadline δ_i^{max}: Tasks generated by vehicles become more latency-sensitive when faced with a smaller maximum deadline $\delta_i^{\max}$. Fig. 10(a) illustrates the average cost of MLDPBA algorithm and other benchmark methods under different maximum deadlines $\delta_i^{\max}$. An increase in the maximum deadline $\delta_i^{\max}$ leads to a reduction in the average cost, as it allows vehicles more time to complete tasks efficiently. The average cost of all algorithms decreases as the maximum deadline increases. Specifically, MLDPBA algorithm reduces the average cost by up to 66.29%, 66.22%, 73.06%, 74.24%, and 74.47% as the maximum deadline ranges from 1.5 to 2.7. Fig. 10(b) illustrates the average satisfaction rate of MLDPBA algorithm and other benchmark methods under different maximum deadlines $\delta_i^{\max}$. As the maximum deadline $\delta_i^{\max}$ increases, the average satisfaction rate of all algorithms improves, indicating a fewer dropped tasks and a lower penalties. The graph clearly demonstrates that the average satisfaction rate of all algorithms increases as the maximum deadline increases. Remarkably, the average satisfaction rate of MLDPBA, MLDBA, SLDBA, GBA, RBA, and LBA algorithms are approximately 85.36%, 83.89%, 82.74%, 71.21%, 60.18%, and 41.62% across the maximum deadline range of 1.5 to 2.7. Hence, it is evident that a greater maximum deadline per task leads to a higher average satisfaction rate.

VII. Conclusion

In this work, we study an MEC-enabled VEN, where each vehicle can offload tasks to edge/cloud computing servers via V2I links or to other end-vehicles via V2V links. We focus on the joint optimization of task offloading, scheduling, and resource allocation. To solve the problem, we propose a MLDPBA, where each vehicle constructs and trains three modules to make different layers' decisions. We provide the detailed algorithm design and perform extensive simulations to evaluate its performance. Simulation results show that our proposed algorithm outperforms the existing algorithms in the literature, and can reduce the average cost by 25.86%–72.51% and increase the average satisfaction rate by 3.48%–90.53%.

Authorized licensed use limited to: Eotvos Lorand University Budapest. Downloaded on October 27,2026 at 13:00:42 UTC from IEEE Xplore. Restrictions apply.

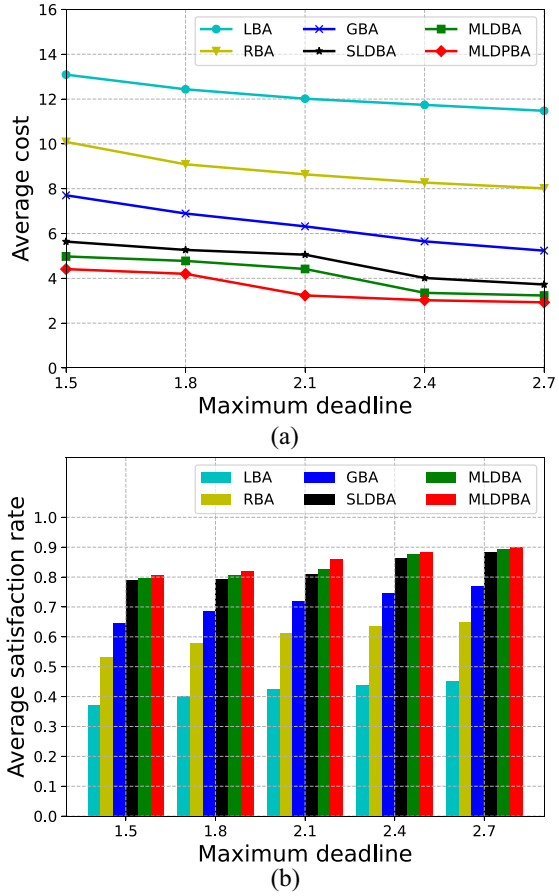


Fig. 10. Performance evaluation under different maximum deadlines $\delta_i^{\max}$. (a) Average cost. (b) Average satisfaction rate.

In future work, we plan to explore more complex network architectures, such as crossroad or grid-like road sections. Additionally, we will aim to leverage communication-based multiagent DRL methods.

APPENDIX

A. Proof for Theorem 1

Proof: To facilitate the analysis of the computing resource allocation problem, we define the coefficients α_1, α_2, and α_3 as follows:

$$\alpha_1 = \varphi_1 \sum_{n=1}^{\tilde{N}_{i,k}} \left(t_1^{\text{loc}} - \tau_{i,k_n} \right), \quad \alpha_2 = \varphi_1 \sum_{n=1}^{\tilde{N}_{i,k}} \left(\sum_{n'=1}^{n} c_{i,k_{n'}} + c_{i,k} \right)$$

$$\alpha_3 = \varphi_2 \xi_i \left(\sum_{n=1}^{\tilde{N}_{i,k}} c_{i,k_n} + c_{i,k} \right).$$

Utilizing these definitions, we can reformulate the objective function of problem (23) as

$$G(f) = \alpha_1 + \frac{\alpha_2}{f} + \alpha_3 f^2.$$

To determine the properties of $G(f)$ and find its minimum, we compute its first and second derivatives with respect to f

$$\frac{dG}{df} = -\frac{\alpha_2}{f^2} + 2\alpha_3 f \tag{26}$$

$$\frac{d^2G}{d^2f} = \frac{2\alpha_2}{f^3} + 2\alpha_3. \tag{27}$$

Since both α_2 and α_3 are positive, it can be seen from (27) that $(d^2G/d^2f) > 0$ holds over the domain $[0, \eta_1 f_i^{\max}]$, which yields that function $G(f)$ is convex. In order to find the optimal solution of function $G(f)$, we set (26) equal to zero, i.e.,

$$\frac{dG}{df} = -\frac{\alpha_2}{f^2} + 2\alpha_3 f = 0.$$

This leads to the optimal computing capacity

$$f_* = \sqrt{\frac{\alpha_2}{2\alpha_3}}.$$

Considering the domain constraint $[0, \eta_1 f_i^{\max}]$, the closed-form solution is determined as

$$f_{i,k}^{\text{HPC}} = \min\left\{ \eta_1 f_i^{\max}, \sqrt{\frac{\alpha_2}{2\alpha_3}} \right\}.$$

■

REFERENCES

[1] J. Wu, Z. Ye, H. Lin, T. Wang, and L. Gao, "A multi-layer deep reinforcement learning approach for joint task offloading and scheduling in vehicular edge networks," in *Proc. IEEE Int. Conf. Commun. (ICC)*, 2023, pp. 3872–3877.

[2] G. S. Park, R. H. Kim, and H. Song, "Collaborative virtual 3-D object modeling for mobile augmented reality streaming services over 5G networks," *IEEE Trans. Mobile Comput.*, vol. 22, no. 7, pp. 3855–3869, Jul. 2023.

[3] Z. Wang, M. Xu, N. Ye, F. Xiao, R. Wang, and H. Huang, "Computer vision-assisted 3-D object localization via COTS RFID devices and a monocular camera," *IEEE Trans. Mobile Comput.*, vol. 20, no. 3, pp. 893–908, Mar. 2021.

[4] D. Jiang et al., "QoE-aware efficient content distribution scheme for satellite-terrestrial networks," *IEEE Trans. Mobile Comput.*, vol. 22, no. 1, pp. 443–458, Jan. 2023.

[5] C. Jiang, Z. Luo, L. Gao, and J. Li, "A truthful incentive mechanism for movement-aware task offloading in crowdsourced mobile edge computing systems," *IEEE Internet Things J.*, vol. 11, no. 10, pp. 18292–18305, May 2024.

[6] Z. Wang, L. Gao, T. Wang, and J. Luo, "Monetizing edge service in mobile Internet ecosystem," *IEEE Trans. Mobile Comput.*, vol. 21, no. 5, pp. 1751–1765, May 2022.

[7] T. Zhou, X. Zeng, D. Qin, N. Jiang, X. Nie, and C. Li, "Cost-aware computation offloading and resource allocation in ultradense multicell, multiuser and multitask MEC networks," *IEEE Trans. Veh. Technol.*, vol. 73, no. 5, pp. 6642–6657, May 2024.

[8] H. Zhang, X. Liu, Y. Xu, D. Li, C. Yuen, and Q. Xue, "Partial offloading and resource allocation for MEC-assisted vehicular networks," *IEEE Trans. Veh. Technol.*, vol. 73, no. 1, pp. 1276–1288, Jan. 2024.

[9] A. Bozorgchenani, S. Maghsudi, D. Tarchi, and E. Hossain, "Computation offloading in heterogeneous vehicular edge networks: Online and off-policy bandit solutions," *IEEE Trans. Mobile Comput.*, vol. 21, no. 12, pp. 4233–4248, Dec. 2022.

[10] X. Wang, Z. Ning, S. Guo, and L. Wang, "Imitation learning enabled task scheduling for online vehicular edge computing," *IEEE Trans. Mobile Comput.*, vol. 21, no. 2, pp. 598–611, Feb. 2022.

[11] W. Miao, G. Min, X. Zhang, Z. Zhao, and J. Hu, "Performance modeling and quantitative analysis of vehicular edge computing with bursty task arrivals," *IEEE Trans. Mobile Comput.*, vol. 22, no. 2, pp. 1129–1142, Feb. 2023.

[12] Y. Li, L. Li, and P. Fan, "Mobility-aware computation offloading and resource allocation for NOMA MEC in vehicular networks," *IEEE Trans. Veh. Technol.*, vol. 73, no. 8, pp. 11934–11948, Aug. 2024.

[13] M. Z. Alam and A. Jamalipour, "Multiagent DRL-based Hungarian algorithm (MADRLHA) for task offloading in multiaccess edge computing Internet of Vehicles (IoVs)," *IEEE Trans. Wireless Commun.*, vol. 21, no. 9, pp. 7641–7652, Sep. 2022.

Authorized licensed use limited to: Eotvos Lorand University Budapest. Downloaded on October 27,2026 at 13:00:42 UTC from IEEE Xplore. Restrictions apply.

[14] Y. Dai, D. Xu, S. Maharjan, and Y. Zhang, "Joint load balancing and offloading in vehicular edge computing and networks," *IEEE Internet Things J.*, vol. 6, no. 3, pp. 4377–4387, Jun. 2019.
[15] K. Tan, L. Feng, G. Dán, and M. Törngren, "Decentralized convex optimization for joint task offloading and resource allocation of vehicular edge computing systems," *IEEE Trans. Veh. Technol.*, vol. 71, no. 12, pp. 13226–13241, Dec. 2022.
[16] H. Cho, Y. Cui, and J. Lee, "Energy-efficient cooperative offloading for edge computing-enabled vehicular networks," *IEEE Trans. Wireless Commun.*, vol. 21, no. 12, pp. 10709–10723, Dec. 2022.
[17] Z. Sun, G. Sun, Y. Liu, J. Wang, and D. Cao, "BARGAIN-MATCH: A game theoretical approach for resource allocation and task offloading in vehicular edge computing networks," *IEEE Trans. Mobile Comput.*, vol. 23, no. 2, pp. 1655–1673, Feb. 2024.
[18] H. Xiao, J. Zhao, J. Feng, L. Liu, Q. Pei, and W. Shi, "Joint optimization of security strength and resource allocation for computation offloading in vehicular edge computing," *IEEE Trans. Wireless Commun.*, vol. 22, no. 12, pp. 8751–8765, Dec. 2023.
[19] Y. Sun et al., "Adaptive learning-based task offloading for vehicular edge computing systems," *IEEE Trans Veh. Technol.*, vol. 68, no. 4, pp. 3061–3074, Apr. 2019.
[20] J. Su, Z. Liu, Y.-A. Xie, Y. Li, K. Ma, and X. Guan, "UEE-delay balanced online resource optimization for cooperative MEC-enabled task offloading in dynamic vehicular networks," *IEEE Internet Things J.*, vol. 11, no. 7, pp. 11496–11507, Apr. 2024.
[21] Y. Yue and J. Wang, "Lyapunov-based dynamic computation offloading optimization in heterogeneous vehicular networks," in *Proc. IEEE Int. Symp. Prod. Compliance Eng. Asia (ISPCE-ASIA)*, 2022, pp. 1–6.
[22] X. Hou et al., "Reliable computation offloading for edge-computing-enabled software-defined IoV," *IEEE Internet Things J.*, vol. 7, no. 8, pp. 7097–7111, Aug. 2020.
[23] H. Wang, T. Lv, Z. Lin, and J. Zeng, "Energy-delay minimization of task migration based on game theory in MEC-assisted vehicular networks," *IEEE Trans. Veh. Technol.*, vol. 71, no. 8, pp. 8175–8188, Aug. 2022.
[24] X. Q. Pham, T. Huynh-The, E. N. Huh, and D. S. Kim, "Partial computation offloading in parked vehicle-assisted multiaccess edge computing: A game-theoretic approach," *IEEE Trans. Veh. Technol.*, vol. 71, no. 9, pp. 10220–10225, Sep. 2022.
[25] N. C. Luong et al., "Applications of deep reinforcement learning in communications and networking: A survey," *IEEE Commun. Surveys Tuts.*, vol. 21, no. 4, pp. 3133–3174, 4th Quart., 2019.
[26] H. Shen, Y. Tian, T. Wang, and G. Bai, "Slicing-based task offloading in space-air-ground integrated vehicular networks," *IEEE Trans. Mobile Comput.*, vol. 23, no. 5, pp. 4009–4024, May 2024.
[27] J. Shi, J. Du, Y. Shen, J. Wang, J. Yuan, and Z. Han, "DRL-based V2V computation offloading for blockchain-enabled vehicular networks," *IEEE Trans. Mobile Comput.*, vol. 22, no. 7, pp. 3882–3897, Jul. 2023.
[28] L. Liang, H. Ye, and G. Y. Li, "Spectrum sharing in vehicular networks based on multiagent reinforcement learning," *IEEE J. Sel. Areas Commun.*, vol. 37, no. 10, pp. 2282–2292, Oct. 2019.
[29] Q. Luo, C. Li, T. H. Luan, and W. Shi, "Collaborative data scheduling for vehicular edge computing via deep reinforcement learning," *IEEE Internet Things J.*, vol. 7, no. 10, pp. 9637–9650, Oct. 2020.
[30] X. Zhang, M. Peng, S. Yan, and Y. Sun, "Deep-reinforcement-learning-based mode selection and resource allocation for cellular V2X communications," *IEEE Internet Things J.*, vol. 7, no. 7, pp. 6380–6391, Jul. 2020.
[31] M. Tang and V. W. S. Wong, "Deep reinforcement learning for task offloading in mobile edge computing systems," *IEEE Trans. Mobile Comput.*, vol. 21, no. 6, pp. 1985–1997, Jun. 2022.
[32] J. Luo, J. Song, F. C. Zheng, L. Gao, and T. Wang, "User-centric UAV deployment and content placement in cache-enabled multi-UAV networks," *IEEE Trans. Veh. Technol.*, vol. 71, no. 5, pp. 5656–5660, May 2022.
[33] T. Wang, Y. Shen, L. Gao, Y. Jiang, X. Zhu, and F. C. Zheng, "Long-term energy consumption and transmission delay tradeoff in wireless-powered body area networks," *IEEE Internet Things J.*, vol. 9, no. 6, pp. 4051–4064, Mar. 2022.
[34] J. Wu, H. Lin, H. Liu, and L. Gao, "A deep reinforcement learning approach for collaborative mobile edge computing," in *Proc. IEEE Int. Conf. Commun. (ICC)*, 2022, pp. 601–606.
[35] W. Fan, J. Liu, M. Hua, F. Wu, and Y. Liu, "Joint task offloading and resource allocation for multiaccess edge computing assisted by parked and moving vehicles," *IEEE Trans. Veh. Technol.*, vol. 71, no. 5, pp. 5314–5330, May 2022.
[36] L. Tan, Z. Kuang, L. Zhao, and A. Liu, "Energy-efficient joint task offloading and resource allocation in OFDMA-based collaborative edge computing," *IEEE Trans. Wireless Commun.*, vol. 21, no. 3, pp. 1960–1972, Mar. 2022.
[37] S. V. Balkus, H. Wang, B. D. Cornet, C. Mahabal, H. Ngo, and H. Fang, "A survey of collaborative machine learning using 5G vehicular communications," *IEEE Commun. Surveys Tuts.*, vol. 24, no. 2, pp. 1280–1303, 2nd Quart., 2022.
[38] T. Ren et al., "Enabling efficient scheduling in large-scale UAV-assisted mobile-edge computing via hierarchical reinforcement learning," *IEEE Internet Things J.*, vol. 9, no. 10, pp. 7095–7109, May 2022.
[39] Y. Qian, R. Wang, J. Wu, B. Tan, and H. Ren, "Reinforcement learning-based optimal computing and caching in mobile edge network," *IEEE J. Sel. Areas Commun.*, vol. 38, no. 10, pp. 2343–2355, Oct. 2020.
[40] L. Huang, S. Bi, and Y. J. A. Zhang, "Deep reinforcement learning for online computation offloading in wireless powered mobile-edge computing networks," *IEEE Trans. Mobile Comput.*, vol. 19, no. 11, pp. 2581–2593, Nov. 2020.
[41] L. Wang, K. Wang, C. Pan, W. Xu, N. Aslam, and A. Nallanathan, "Deep reinforcement learning based dynamic trajectory control for UAV-assisted mobile edge computing," *IEEE Trans. Mobile Comput.*, vol. 21, no. 10, pp. 3536–3550, Oct. 2022.
[42] Z. Wang, T. Schaul, and M. Hessel, "Dueling network architectures for deep reinforcement learning," in *Proc. 33rd Int. Conf. Mach. Learn. (PMLR)*, 2016, pp. 1995–2003.
[43] M. Hausknecht and P. Ston, "Deep reinforcement learning in parameterized action space," 2015, *arXiv:1511.04143*.
[44] S. Zhang, H. Gu, K. Chi, L. Huang, K. Yu, and S. Mumtaz, "DRL-based partial offloading for maximizing sum computation rate of wireless powered mobile edge computing network," *IEEE Trans. Wireless Commun.*, vol. 21, no. 12, pp. 10934–10948, Dec. 2022.
[45] C. Chen, Y. Zeng, H. Li, Y. Liu, and S. Wan, "A multihop task offloading decision model in MEC-enabled Internet of Vehicles," *IEEE Internet Things J.*, vol. 10, no. 4, pp. 3215–3230, Feb. 2023.
[46] Y. S. Nasir and D. Guo, "Multiagent deep reinforcement learning for dynamic power allocation in wireless networks," *IEEE J. Sel. Areas Commun.*, vol. 37, no. 10, pp. 2239–2250, Oct. 2019.



Jiaqi Wu (Graduate Student Member, IEEE) received the master's degree from Guangxi University, Nanning, China, in 2020. He is currently pursuing the Ph.D. degree with the School of Electronics and Information Engineering, Harbin Institute of Technology, Shenzhen, China.

His main research interests are in the interdisciplinary area between network optimization and machine learning, with particular focuses on the deep reinforcement learning, mobile edge computing, large language models, and AI-generated contents.



Ming Tang (Member, IEEE) received the Ph.D. degree from The Chinese University of Hong Kong, Hong Kong, in 2018.

She is currently an Associate Professor with the Department of Computer Science and Engineering, Southern University of Science and Technology, Shenzhen, China. She was a Postdoctoral Research Fellow with The University of British Columbia, Vancouver, BC, Canada, from 2018 to 2021. Her research interests include mobile edge computing, federated learning, and network economics.

Dr. Tang received the 2022 Best Paper Award from IEEE TRANSACTIONS ON MOBILE COMPUTING. She recently serves as an Associate Editor for IEEE INTERNET OF THINGS JOURNAL and IEEE OPEN JOURNAL OF THE COMMUNICATIONS SOCIETY.

Authorized licensed use limited to: Eotvos Lorand University Budapest. Downloaded on October 27,2026 at 13:00:42 UTC from IEEE Xplore. Restrictions apply.



Changkun Jiang (Member, IEEE) received the Ph.D. degree in information engineering from The Chinese University of Hong Kong, Hong Kong, in 2017.

He is currently an Assistant Professor with the College of Computer Science and Software Engineering, Shenzhen University, Shenzhen, China. His main research interests lie in the area of networked systems and computing, with a particular emphasis on cloud–edge–client collaborative network resource management using analytical tools, such as network games, mechanism design, optimization, and learning. More recently, he has also been interested in artificial intelligence and its applications in several related interdisciplinary areas.



Lin Gao (Senior Member, IEEE) received the Ph.D. degree in electronic engineering from Shanghai Jiao Tong University, Shanghai, China, in 2010.

He is currently a Professor with the School of Electronics and Information Engineering, Harbin Institute of Technology, Shenzhen, China. His research interests are in the interdisciplinary area between network optimizations, game theory, and artificial intelligence, with particular focus on mobile-edge intelligence, federated learning, and large language models.

Prof. Gao is the co-recipient of four Best Paper Awards from leading conference, Proceedings on Wireless Networking. He received the IEEE ComSoc Asia–Pacific Outstanding Young Researcher Award in 2016.



Bin Cao (Member, IEEE) received the Ph.D. degree in information and communication engineering from Harbin Institute of Technology, Shenzhen, China, in 2013.

He is currently an Associate Professor with the School of Electronics and Information Engineering, Harbin Institute of Technology. From 2010 to 2012, he was a Visiting Scholar with the University of Waterloo, Waterloo, ON, Canada. His research interests include signal processing for wireless communications, cognitive radio networking, and resource allocation for wireless networks.

Authorized licensed use limited to: Eotvos Lorand University Budapest. Downloaded on October 27,2026 at 13:00:42 UTC from IEEE Xplore. Restrictions apply.